

Software Testing, Debugging, and Maintenance

RUBING HUANG (黃如兵)

Email: rbhuang@must.edu.mo

Office: A209

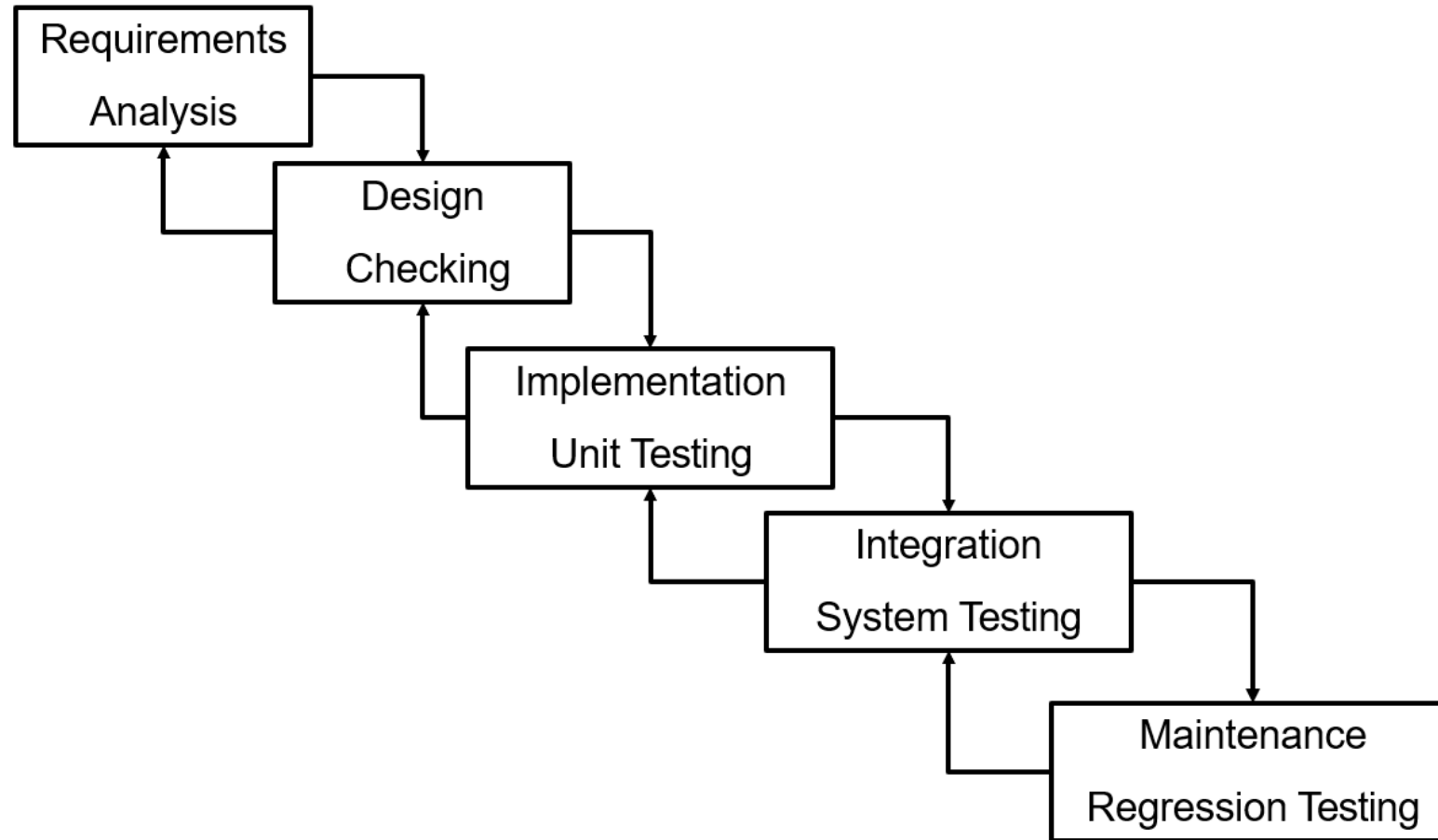
Homepage: <https://huangrubing.github.io>

Software Testing

Validation & Verification (V&V)

- Validation: “process of evaluating a system or component during or at the **end of the development process** to determine whether it satisfies **specified requirements**”.
 - ✓ Are we building the **right** product?
- Verification: “process of evaluating a system or component to determine whether the products of **a given development phase** satisfy the **conditions imposed at the start of that phase**”.
 - ✓ Are we building the product **right**?

V&V in Modified Waterfall Model



Why Conduct Testing?

- Many tragedies because of software bugs:
 - ✓ 1963, America, Space Rocket Explosion, losing \$ 10 million, due to “DO 5 I = 1, 3” was written as “DO 5 I = 1.3”;
 - ✓ 1996, European Ariane 5 Explosion, losing \$ 7 billion, due to “an overflow error to try to store a 64-bit number into a 16-bit space”;
 - ✓ 2000, Intel, losing \$ 400 million, due to “use two bits to save the year”.
 - ✓ 2011, 12306 website, crash, due to “many users at the same time”;
 - ✓ 2019, Hongkong train accident, due to “some source code is modified to bring bugs”;
 - ✓ ...

Why Conduct Testing?

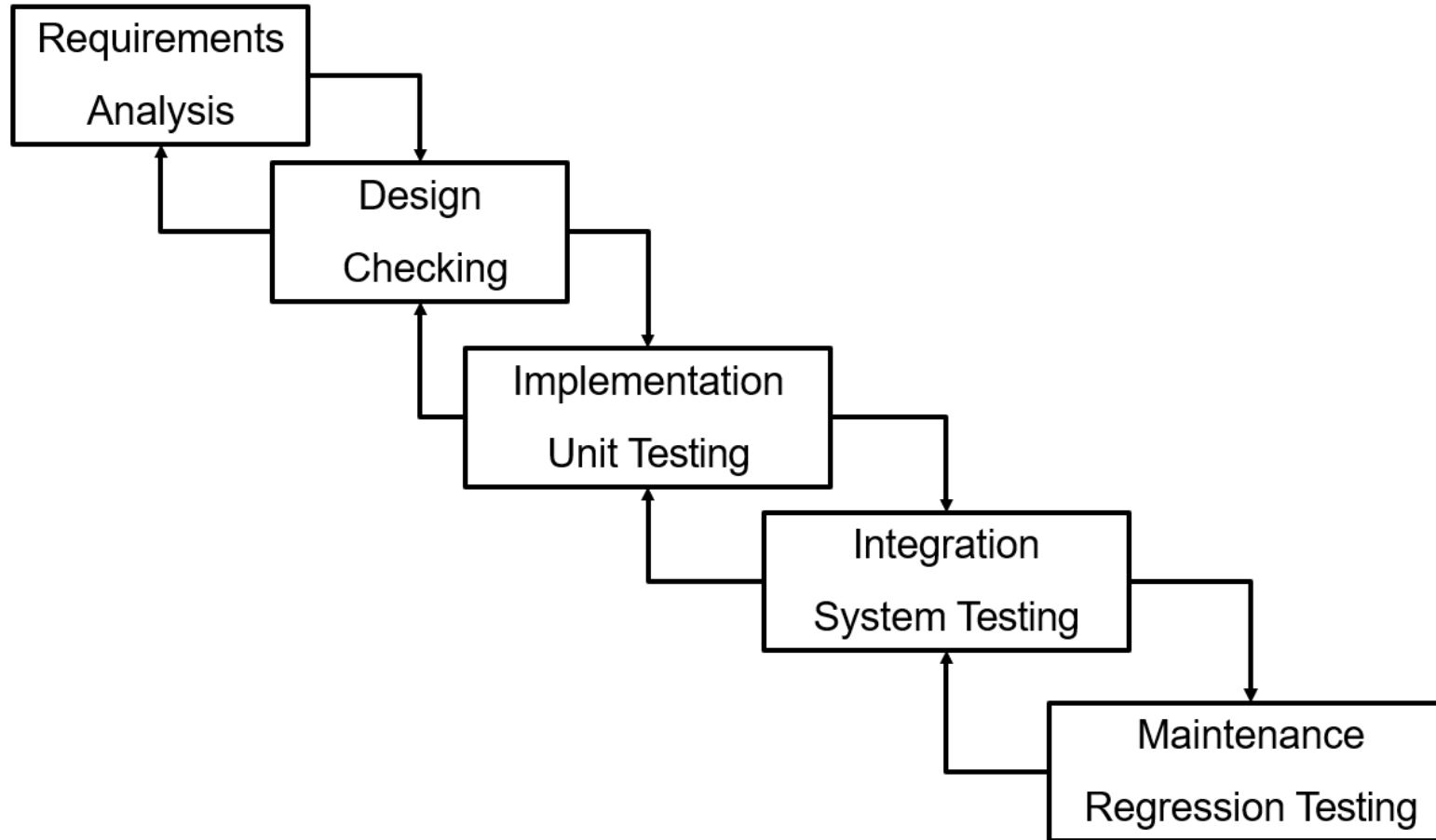
- Improve quality - find faults
- Measure quality
 - ✓ Prove there are no faults? (Is it possible?).
 - ✓ Determine whether software is ready to be released.
 - ✓ Determine what to work on.
- Learn the software

"Testing shows the presence, not the absence of bugs."
1972 ACM Turing Awardee: Edsger W. Dijkstra

Who Write Tests?

- Developer?
- Separate “quality assurance” group?
- User?
- Someone with a degree in “testing”?
- Someone with domain knowledge of the software?

When to Write Tests?



When to Run Tests?

- **After** you change the software under test.
- **Before** you deliver the software.
 - ✓ To customer;
 - ✓ To next phase of project.

When to Stop Testing?

- There are the following criteria to stop testing:
 - ✓ Testing exceeds the given time;
 - ✓ Executing all test cases without finding bugs;
 - ✓ Adopting a specific test case design approach;
 - ✓ Identifying a certain number of bugs;
 - ✓ Identifying a certain number of bugs within a given time.

Some Misunderstandings to Testing

- More time does more testing, and less time does less testing;
- Software testers do software testing, and developers do not care;
- Design-implementation-testing, so testing is followed by implementation;
- If software has bugs, testers' responsibility;
- Testing is easier than coding;
- Everyone can do testing.

Criteria to Software Testing

- All testing should track back to users' requirements;
- Before testing, should conduct testing plan;
- Pareto principle: 80% of faults may be caused by 20% of code;
- Testing from “small scale” to “large scale”;
- Exhaustive testing is impossible;
- To achieve the “best testing”, the independent third-party should do testing.

Classification of Software Testing

- Different viewpoints achieve different categories.
- From the perspective of whether code is available:
 - ✓ Black-box testing;
 - ✓ White-box testing;
 - ✓ Gray-box testing (model-based testing).

Classification of Software Testing

- Different viewpoints achieve different categories.
- From the perspective of whether software is executed:
 - ✓ Dynamic testing [at run time];
 - ✓ Static testing [at compile time].

Classification of Software Testing

- Different viewpoints achieve different categories.
- From the perspective of different testing levels (or stages):
 - ✓ Unit testing;
 - ✓ Integration testing;
 - ✓ System testing;
 - ✓ Regression testing;
 - ✓ Validation testing.

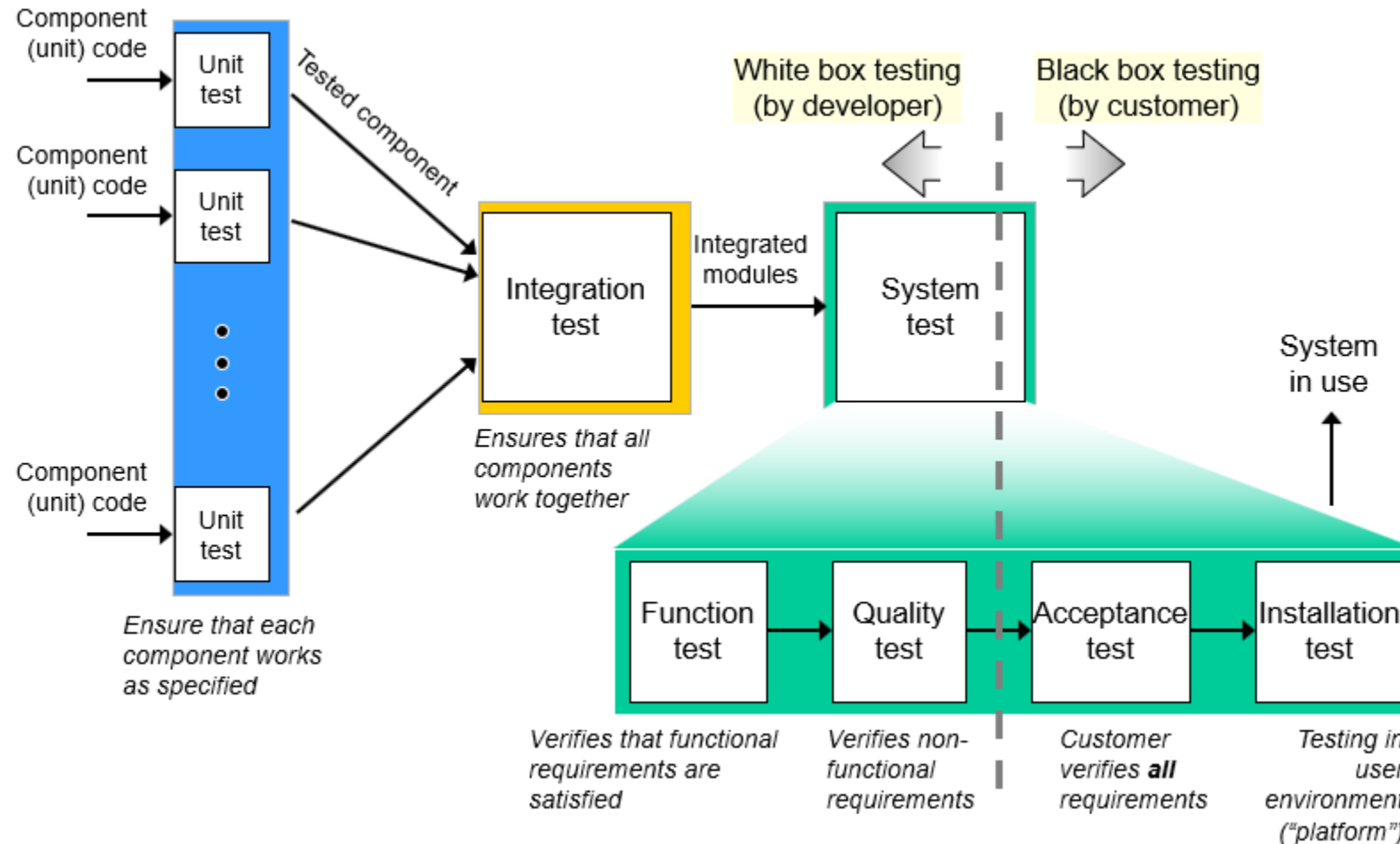
Classification of Software Testing

- Different viewpoints achieve different categories.
- From the perspective of whether testing is active or passive:
 - ✓ Active testing;
 - ✓ Passive testing.

Classification of Software Testing

- Different viewpoints achieve different categories.
- From the perspective of a specific target:
 - ✓ Security testing;
 - ✓ Reliability testing;
 - ✓ Stress testing;
 - ✓ Compatibility Testing;
 - ✓ ...

Logical Organization of Testing



Career for Software Testing

- Software testing engineer.
- Software testing developer.
- Software maintenance engineer (regression testing).

Test Case

What is a Test Case?

- Aka: test (for short).
- Run software under test with known inputs (**test inputs**/data), check results (i.e., **test oracles**).
 - ✓ Tests pass or fail
- Tests can document faults.
- Tests can document code.

A Simple Example

- Use C programming language to write source code for calculating $Ax^2 + Bx + C = 0$ with given three parameters A , B , and C .
- What is a test case for this simple example?

A Simple Example

- Use C++ or Java (OO programming language) to implement the same function as above.
- What is a test case for this simple example?

Manual Test: Example

Test ID	Description	Expected Results	Actual Results
addInventorySuccessfully	Precondition: Run CoffeeMaker Click Menu option 4, "Add inventory";	Newly added inventory displayed on the main menu page	
	Enter Coffee: 5 Milk: 3 Sugar: 7 Chocolate: 2;		
	Return to the main menu page.		

Automated Test: Example

- **Doubling** the balance and then plus 10
- Where is test input? Where is test oracle?

```
public int calAmount ()  
{  
    int ret = balance * 3;  
    ret = ret + 10;  
    return ret;  
}
```

```
void testCalAmount()  
{  
    Account a = new Account();  
    Account.setBalance(0);  
    int amount = Account.calAmount();  
    assertTrue(amount == 10);  
}
```

Test Automation

- Tests are code/scripts.
 - ✓ Created by a tool that records user actions;
 - ✓ Written manually in a test scripting language.
- Real projects can often have more test code than production code.
- Test code is “boring”.
 - ✓ Build some (complex) data values
 - ✓ Run a method or multiple methods
 - ✓ Check the result

When to Use Automated Tests vs. Manual Tests

- Automated Tests
 - ✓ When you run them over and over (Manual regression testing is BORING)
 - As you build the software incrementally
 - As you change it
 - ✓ When you want to use them as documentation (example runs)
- Manual Tests (good for exploratory, GUI testing)
 - ✓ When you are only going to run the test once
 - Should you automate the second time you run?
 - ✓ When tester doesn't know how to program
 - ... or maintainer doesn't know how to program
 - ✓ When tests are too expensive to automate
 - ... and maintain automated tests

What Kind of Tests?

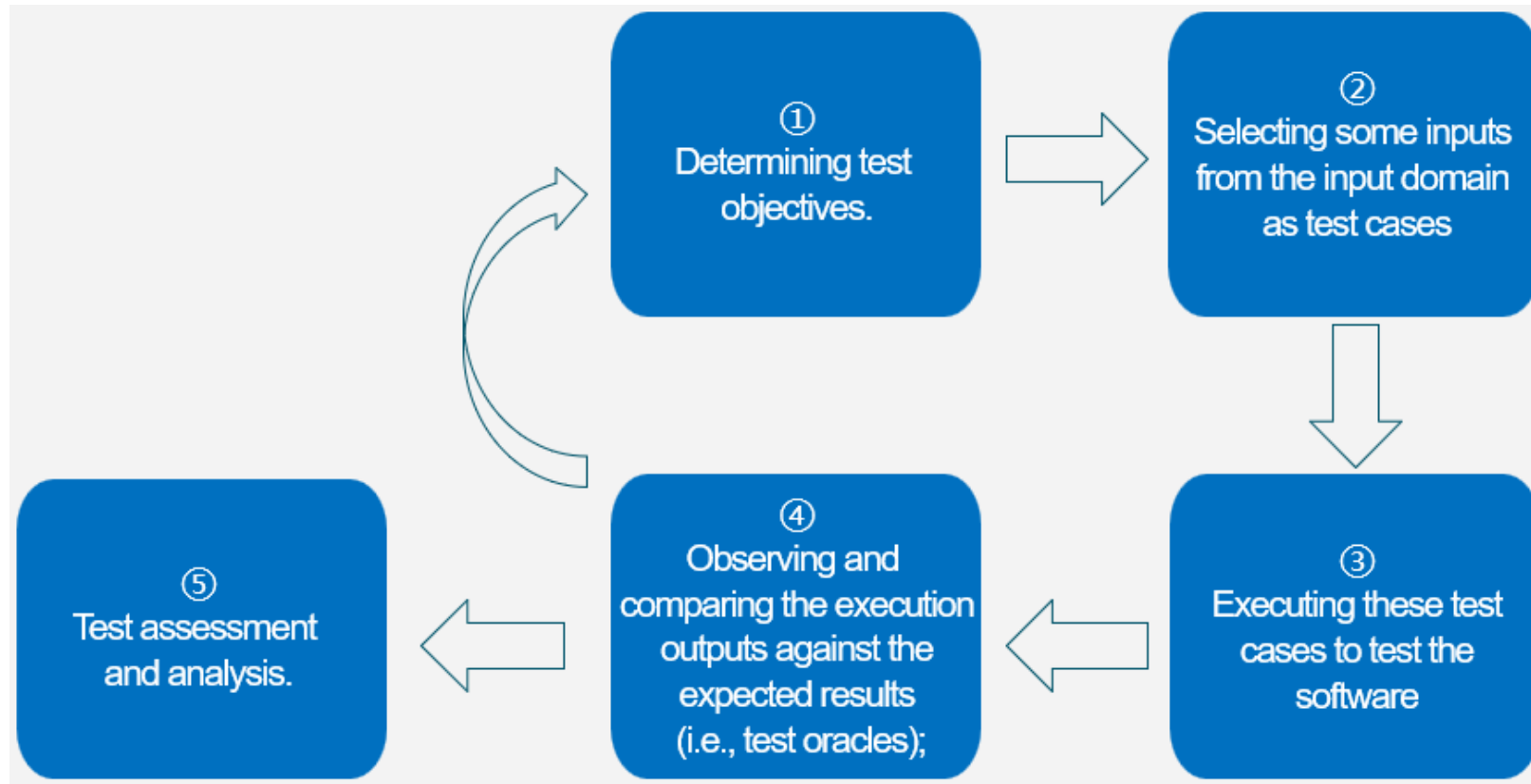
- Programmer tests / Non-programmer tests
- Developer / Tester
- Unit tests / Integration tests / System tests
- Automated tests / Manual tests
- Regression tests / Exploratory tests

More Kinds of Tests

- Performance tests
 - ✓ Measure time of each test
- Load tests
 - ✓ Test system under heavy load
- Usability tests
 - ✓ See how well people can use the software
- Security tests ...

Test Activities

Testing Procedure



Test Activities

Test ID	Description	Expected Results	Actual Results
addInventorySuccessfully	Precondition: Run CoffeeMaker	Newly added inventory displayed on the main menu page	
	Click Menu option 4, "Add inventory";		
	Enter Coffee: 5 Milk: 3 Sugar: 7 Chocolate: 2;		
	Return to the main menu page.		

- 1. Test Design
- 2. Test Automation
- 3. Test Execution
- 4. Test Evaluation

Test Design

Test ID	Description	Expected Results	Actual Results
addInventorySuccessfully	Precondition: Run CoffeeMaker Click Menu option 4, "Add inventory";	Newly added inventory displayed on the main menu page	
	Enter Coffee: 5 Milk: 3 Sugar: 7 Chocolate: 2;		
	Return to the main menu page.		

- Design test inputs to satisfy a test objective

Test Automation

Test ID	Description	Expected Results	Actual Results
addInventorySuccessfully	Precondition: Run CoffeeMaker Click Menu option 4, "Add inventory";	Newly added inventory displayed on the main menu page	
	Enter Coffee: 5 Milk: 3 Sugar: 7 Chocolate: 2;		
	Return to the main menu page.		

- Embed test inputs into executable scripts

Test Execution

Test ID	Description	Expected Results	Actual Results
addInventorySuccessfully	Precondition: Run CoffeeMaker Click Menu option 4, "Add inventory";	Newly added inventory displayed on the main menu page	
	Enter Coffee: 5 Milk: 3 Sugar: 7 Chocolate: 2;		
	Return to the main menu page.		

- Run test inputs on the software and record actual results

Test Evaluation

Test ID	Description	Expected Results	Actual Results
addInventorySuccessfully	Precondition: Run CoffeeMaker Click Menu option 4, "Add inventory";	Newly added inventory displayed on the main menu page	
	Enter Coffee: 5 Milk: 3 Sugar: 7 Chocolate: 2;		
	Return to the main menu page.		

- Evaluate actual results / document expected results

Assigning Right Persons

- Test Design
 - ✓ technical knowledge of **testing/programming**
- Test Automation
 - ✓ knowledge of **scripting**
- Test Execution
 - ✓ very **little** knowledge
- Test Evaluation
 - ✓ **domain** knowledge

Two Types of Programs

- Software (system or program) under test (SUT, 被测程序)
- Test-driven program (测试驱动程序), which drives the testing process, including executing test cases for SUT, collecting outputs, determining whether test is pass or not by comparing actual and expected outputs, and displaying results using table, graph, or else.

Challenges of Testing

- Some challenges:
 - ✓ Test case generation (efficacy and efficiency?)
 - ✓ Test oracle (how about no test oracles?)
 - Google search example
 - Calculate $\sin(x)$ example
 - ML or DL applications example
 - ✓ Test-driven programs writing (another process of software development, no bugs?)
 - ✓ Automated (totally automated?)

Relating Fault and Failure

Mistake, Fault, Error, Failure

What does *Bug* mean in "*Bug Report*"?

Developer makes a **mistake**.

Fault (**defect, bug**) introduced in the software.

Fault remains undetected during testing

i.e., run **test inputs** & check **outputs**

The program **fails** (based on **test oracles**) during execution

i.e., it behaves unexpectedly

Error: difference between (1) computed, observed, or measured value/condition & (2) true, specified, or theoretically correct value/condition

Mistake, Fault, Error, Failure

- Fault -- often referred to as Bug or Defect
 - ✓ A static defect in software (incorrect lines of code)
- Error
 - ✓ An incorrect internal state (**unobserved**), referred to as Mistake in some references
- Failure
 - ✓ External, incorrect behavior with respect to the expected behavior (**observed**)

Mistake, Fault, Error, Failure

- Saying “failure” means you know something is wrong but don't know the cause.
- Saying “fault” means you know the cause, but don't know why the fault occurred.
- Saying “error” means you know why the fault occurred.

Where is Fault, Error, or Failure?

```
public static int numZero (int[] x) {  
    //Effects: if x==null throw NullPointerException  
    //          else return the number of occurrences of 0 in x  
    int count = 0;  
    for (int i = 1; i < x.length; i++) {  
        if (x[i]==0) {  
            count++;  
        }  
    }  
    return count;  
}
```

Error State:

x = [2,7,0]

i = 1

count = 0

PC=first iteration for

Expected State:

x = [2,7,0]

i = 0

count = 0

PC=first iteration for

Fix: for(int i=0; i<x.length; i++)

x = [2,7,0], fault executed, **error**, no failure

x = [0,7,2], fault executed, error, failure

State of the program: x, i, count, PC

Where is Fault, Error, or Failure?

```
/* Effect: if x==null throw NullPointerException.  
Otherwise, return the index of the last element  
in the array 'x' that equals integer 'y'.  
Return -1 if no such element exists. */  
  
public int findLast (int[] x, int y) {  
    for (int i=x.length-1; i>0; i--) {  
        if (x[i] == y) { return i; }  
    }  
    return -1;  
}  
  
/* test 1: x=[2,3,5], y=2;  
expect: findLast(x,y) == 0  
test 2: x=[2,3,5,2], y=2;  
expect: findLast(x,y) == 3 */
```

Read this faulty program, which includes a test case that results in failure. Answer the following questions.

- (a) Identify the fault, and fix the fault.**
- (b) If possible, identify a test case that does not execute the fault.
- (c) If possible, identify a test case that executes the fault, but does not result in an error state.
- (d) If possible identify a test case that results in an error, but not a failure. Hint: Don't forget about the program counter.
- (e) For the given test case 'test1', identify the first error state. Be sure to describe the complete state.

Where is Fault, Error, or Failure?

- (a) The for-loop should include the 0 index:
 - ✓ `for (int i=x.length-1; i >= 0; i--)`

Where is Fault, Error, or Failure?

```
/* Effect: if x==null throw NullPointerException.  
Otherwise, return the index of the last element  
in the array 'x' that equals integer 'y'.  
Return -1 if no such element exists. */  
  
public int findLast (int[] x, int y) {  
    for (int i=x.length-1; i>0; i--) {  
        if (x[i] == y) { return i; }  
    }  
    return -1;  
}  
  
/* test 1: x=[2,3,5], y=2;  
expect: findLast(x,y) == 0  
test 2: x=[2,3,5,2], y=2;  
expect: findLast(x,y) == 3 */
```

Read this faulty program, which includes a test case that results in failure. Answer the following questions.

- (a) Identify the fault, and fix the fault.
- (b) If possible, identify a test case that does not execute the fault.**
- (c) If possible, identify a test case that executes the fault, but does not result in an error state.
- (d) If possible identify a test case that results in an error, but not a failure. Hint: Don't forget about the program counter.
- (e) For the given test case 'test1', identify the first error state. Be sure to describe the complete state.

Where is Fault, Error, or Failure?

- (b) The null value for x will result in a NullPointerException before the loop test is evaluated, hence no execution of the fault.
 - ✓ Input: x = null; y = 3
 - ✓ Expected Output: NullPointerException
 - ✓ Actual Output: NullPointerException

Where is Fault, Error, or Failure?

```
/* Effect: if x==null throw NullPointerException.  
Otherwise, return the index of the last element  
in the array 'x' that equals integer 'y'.  
Return -1 if no such element exists. */  
  
public int findLast (int[] x, int y) {  
    for (int i=x.length-1; i>0; i--) {  
        if (x[i] == y) { return i; }  
    }  
    return -1;  
}  
  
/* test 1: x=[2,3,5], y=2;  
expect: findLast(x,y) == 0  
test 2: x=[2,3,5,2], y=2;  
expect: findLast(x,y) == 3 */
```

Read this faulty program, which includes a test case that results in failure. Answer the following questions.

- (a) Identify the fault, and fix the fault.
- (b) If possible, identify a test case that does not execute the fault.
- (c) If possible, identify a test case that executes the fault, but does not result in an error state.**
- (d) If possible identify a test case that results in an error, but not a failure. Hint: Don't forget about the program counter.
- (e) For the given test case 'test1', identify the first error state. Be sure to describe the complete state.

Where is Fault, Error, or Failure?

- (c) For any input where y appears in a position that is not position 0, there is no error.
 - ✓ Input: $x = [2, 3, 5]$; $y = 3$;
 - ✓ Expected Output: 1
 - ✓ Actual Output: 1

Also, if x is empty, there is no error.

Where is Fault, Error, or Failure?

```
/* Effect: if x==null throw NullPointerException.  
Otherwise, return the index of the last element  
in the array 'x' that equals integer 'y'.  
Return -1 if no such element exists. */  
  
public int findLast (int[] x, int y) {  
    for (int i=x.length-1; i>0; i--) {  
        if (x[i] == y) { return i; }  
    }  
    return -1;  
}  
  
/* test 1: x=[2,3,5], y=2;  
expect: findLast(x,y) == 0  
test 2: x=[2,3,5,2], y=2;  
expect: findLast(x,y) == 3 */
```

Read this faulty program, which includes a test case that results in failure. Answer the following questions.

- (a) Identify the fault, and fix the fault.
- (b) If possible, identify a test case that does not execute the fault.
- (c) If possible, identify a test case that executes the fault, but does not result in an error state.
- (d) If possible identify a test case that results in an error, but not a failure. Hint: Don't forget about the program counter.**
- (e) For the given test case 'test1', identify the first error state. Be sure to describe the complete state.

Where is Fault, Error, or Failure?

- (d) For an input where y is not in x , the missing path (i.e. an incorrect PC on the final loop that is not taken, normally $i = 2, 1, 0$, but this one has only $i = 2, 1$,) is an error, but there is no failure.
 - ✓ Input: $x = [2, 3, 5]$; $y = 7$;
 - ✓ Expected Output: -1
 - ✓ Actual Output: -1

Where is Fault, Error, or Failure?

```
/* Effect: if x==null throw NullPointerException.  
Otherwise, return the index of the last element  
in the array 'x' that equals integer 'y'.  
Return -1 if no such element exists. */  
  
public int findLast (int[] x, int y) {  
    for (int i=x.length-1; i>0; i--) {  
        if (x[i] == y) { return i; }  
    }  
    return -1;  
}  
  
/* test 1: x=[2,3,5], y=2;  
expect: findLast(x,y) == 0  
test 2: x=[2,3,5,2], y=2;  
expect: findLast(x,y) == 3 */
```

Read this faulty program, which includes a test case that results in failure. Answer the following questions.

- (a) Identify the fault, and fix the fault.
- (b) If possible, identify a test case that does not execute the fault.
- (c) If possible, identify a test case that executes the fault, but does not result in an error state.
- (d) If possible, identify a test case that results in an error, but not a failure. Hint: Don't forget about the program counter.
- (e) For the given test case 'test1', identify the first error state. Be sure to describe the complete state.**

Where is Fault, Error, or Failure?

- (e) Note that the key aspect of the error state is that the PC is outside the loop (following the false evaluation of the $0 > 0$ test. In a correct program, the PC should be at the if-test, with index $i == 0$.
 - ✓ Input: $x = [2, 3, 5]$; $y = 2$;
 - ✓ Expected Output: 0
 - ✓ Actual Output: -1
 - ✓ First Error State:
 - $x = [2, 3, 5]$
 - $y = 2$;
 - $i = 0$ (or undefined);
 - PC = return -1;

Fault & Failure Model: PIE

(Voas, 1992)

- Three conditions necessary/sufficient for a fault → a failure
- **1. Execution**
 - ✓ The fault-containing location(s) in the software is executed.
- **2. Infection**
 - ✓ State of the software is infected, i.e., causing error.
- **3. Propagation**
 - ✓ Infected state is propagated to cause some state/output of the software to be observed (e.g., with test oracle).

Tester vs. Test Manager

(Voas, 1992)

- Test Manager: Looks like the code under test is not achieving 100% statement coverage. Please work hard to achieve 100% or close to 100% statement coverage as much as possible.
- Tester: Hmm... boss, our goal is to detect faults. I don't think I need to spend more efforts to achieve 100% statement coverage.
- Test Manager: Well, according to the PIE model, you need to. The reason is ... [You fill in here]

Tester vs. Test Manager

(Voas, 1992)

- Tester: Boss, following your instruction, I work very hard and I have already achieved 100% statement coverage! I would like to take a vacation in Hawaii. Could you approve?
- Test Manager: Well, according to the PIE model, no! The reason is …..[You fill in here]

Testing & Debugging

- **Testing:** The process of finding inputs (and test oracles) that cause the software to fail.
- **Debugging:** The process of finding a fault given a failure.
- Testing is hard:
 - ✓ Often, only specific inputs will trigger the fault into creating a failure.
- Debugging is hard:
 - ✓ Given a failure, it is often difficult to know the fault.

Testing is Hard

```
if ( x - 100 <= 0 )  
    if ( y - 100 <= 0 )  
        if ( x + y - 200 == 0 )  
            crash();
```

- Only input $x = 100$ & $y = 100$ triggers the crash.
- If x and y are 32-bit integers, what is the probability of a crash?

White-Box Testing

Black-Box vs. White-Box Techniques for Writing Tests

- **White-box (from code)**
 - ✓ Code coverage
- Black-box (from specifications)
 - ✓ Equivalence partitioning
 - ✓ Boundary value analysis

White-box Tests

- Given knowledge of internal workings, thoroughly test what is happening on the inside
 - ✓ Exercise all the code portions (recall the PIE model)
- Logical paths through code are tested
 - ✓ Methods
 - ✓ Statements
 - ✓ Branches/Conditions
 - ✓ Loops

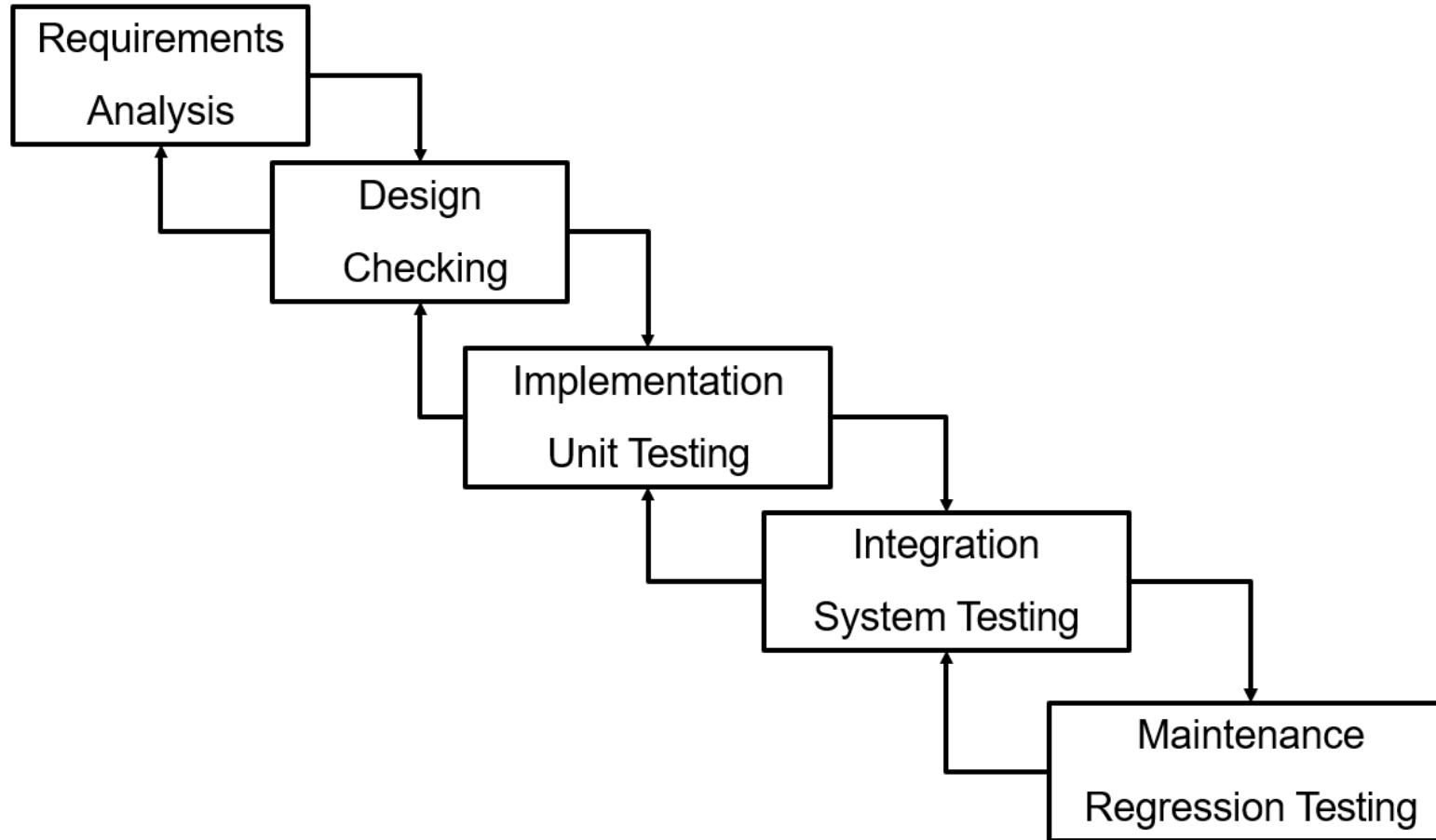
White-box Tests

- Still use requirements to determine the expected behavior
- Large number - take a long time to write
 - ✓ Impossible to thoroughly exercise all paths
 - ✓ Can be practical if a limited number of “important” paths are focused
 - ✓ Can be practical to examine and test important data structures

Guidelines on White-Box Testing

- A metric for assessing how good your test suite is
 - ✓ Various type of code coverage, e.g., statement coverage, branch coverage
- Generation of test cases to cover those not-yet-covered code entities, e.g., statement, branch
- Some black-box testing approaches are also required (such as Equivalence Class/Boundary Value Analysis)
 - ✓ Still applies!
- Typically conducted after conducting black-box testing

When to Conduct White-Box Testing



Code Coverage (Logic + Control Procedure)

- **Logic**

- ✓ Line coverage;
- ✓ Statement coverage;
- ✓ Function or method coverage;
- ✓ Decision or branch coverage;
- ✓ Condition coverage;
- ✓ Decision/condition coverage;
- ✓ Condition combination coverage.

- **Control Procedure**

- ✓ Path coverage.

Line Coverage

- Percentage of source code lines executed by test cases.
 - ✓ For developer easiest to work with
 - ✓ Precise percentage depends on layout?
 - `int x = 10; if (z++ < x) y = x+z;`
- In practice, coverage not based on lines, but on **control flow graph**

Deriving a Control Flow Graph

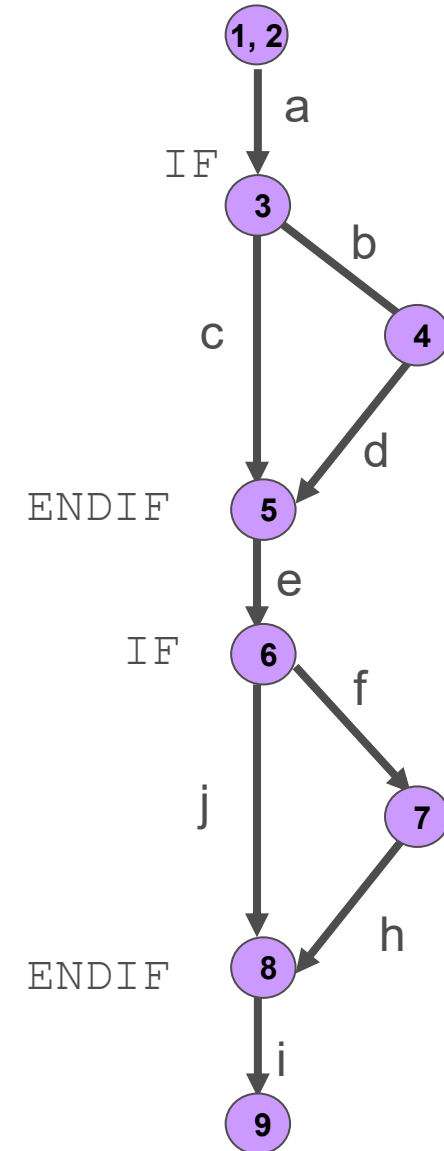
```
1 int x, y;  
2 double z;  
3     if (x > 0 && y > 0) then  
4         z = z / x;  
5     end if  
6     if (x > 1 || z > 1) then  
7         z = z + 1;  
8     end if  
9     z = y + z;
```

Deriving a Control Flow Graph

```
1 int x, y;  
2 double z;  
3     if (x > 0 && y > 0) then  
4         z = z / x;  
5     end if  
6     if (x > 1 || z > 1) then  
7         z = z + 1;  
8     end if  
9     z = y + z;
```

Deriving a Control Flow Graph

```
1 int x, y;  
2 double z;  
3     if (x > 0 && y > 0) then  
4         z = z / x;  
5     end if  
6     if (x > 1 || z > 1) then  
7         z = z + 1;  
8     end if  
9     z = y + z;
```



Statement Coverage

- Adequacy criterion: Each statement (or node in the CFG) must be executed at least once.

- Coverage = $\frac{\text{\# executed statements}}{\text{\# statements}}$

```
1 int x, y;  
2 double z;  
3     if (x > 0 && y > 0) then  
4         z = z / x;  
5     end if  
6     if (x > 1 || z > 1) then  
7         z = z + 1;  
8     end if  
9     z = y + z;
```

(x, y, z) = (1, 1, 2)

Function/Method coverage

- Adequacy criterion: All methods in all classes have been called
- Coverage = $\frac{\# \text{ executed methods}}{\# \text{ methods}}$

Decision or Branch Coverage

- Adequacy criterion: Every branch or decision going out of node executed at least once, i.e., each branch or decision predicate must be both true and false.

```
1  int x, y;  
2  double z;  
3      if (x > 0 && y > 0) then  
4          z = z / x;  
5      end if  
6      if (x > 1 || z > 1) then  
7          z = z + 1;  
8      end if  
9      z = y + z;
```

$(x, y, z) = (1, 1, 2)$

$(x, y, z) = (-1, 1, 0)$

Condition Coverage

- Adequacy criterion: All sub-expression conditions have been true and false.

```
1 int x, y;  
2 double z;  
3     if (x > 0 && y > 0) then  
4         z = z / x;  
5     end if  
6     if (x > 1 || z > 1) then  
7         z = z + 1;  
8     end if  
9     z = y + z;
```

$(x, y, z) = (2, -1, 0)$

$(x, y, z) = (-1, 1, 2)$

Decision/Condition Coverage

- Adequacy criterion: Decision coverage = 100% and condition coverage = 100%, i.e., each branch should be true and false; while each sub-expression conditions have been true and false.

```
1 int x, y;  
2 double z;  
3     if (x > 0 && y > 0) then  
4         z = z / x;  
5     end if  
6     if (x > 1 || z > 1) then  
7         z = z + 1;  
8     end if  
9     z = y + z;
```

$(x, y, z) = (2, 1, 2)$

$(x, y, z) = (-1, 0, 1)$

Condition Combination Coverage

- Adequacy criterion: In each branch, combinations of sub-expression conditions should be incurred. For example, a branch or decision (A && B), result in 4 cases (T, T), (T, F), (F, T), (F, F).

```
1 int x, y;  
2 double z;  
3     if (x > 0 && y > 0) then  
4         z = z / x;  
5     end if  
6     if (x > 1 || z > 1) then  
7         z = z + 1;  
8     end if  
9     z = y + z;
```

(x, y, z) = (2, 1, 6)

(x, y, z) = (2, -1, -2)

(x, y, z) = (-1, 2, 3)

(x, y, z) = (-1, -2, -3)

Infeasible Coverage

- There are parts that cannot be covered by **ANY** test case.

```
for (i=0; i<100; i++)  
{  
    printf("%d\n", i);  
}  
if (i == 200)  
    panic("program internal error: 3");
```

Loop Coverage

- Write test cases such that
 - ✓ Don't go through the loop body at all (0 pass)
 - ✓ Go through the loop body once (1 pass)
 - ✓ Go through the loop body twice (2 passes)
 - ✓ Go through the loop body many times (n passes, $n > 2$)

Path Coverage

- Adequacy criterion: To cover all possible paths in the DFG.

```
1 int x, y;  
2 double z;  
3     if (x > 0 && y > 0) then  
4         z = z / x;  
5     end if  
6     if (x > 1 || z > 1) then  
7         z = z + 1;  
8     end if  
9     z = y + z;
```

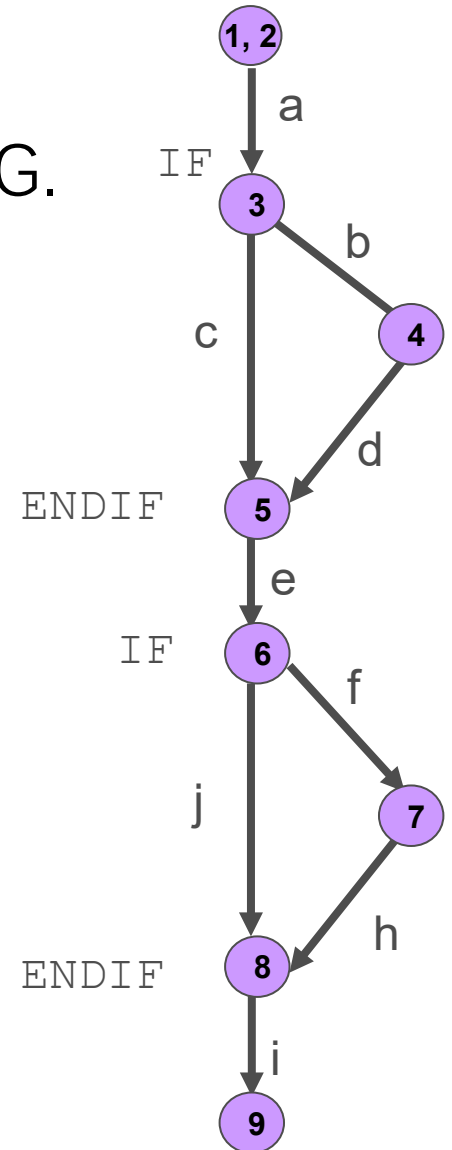
Condition Combination:

$(x, y, z) = (2, 1, 6)$

$(x, y, z) = (2, -1, -2)$

$(x, y, z) = (-1, 2, 3)$

$(x, y, z) = (-1, -2, -3)$



Code Coverage Tool

- Tool “instruments” the program.
- You run your test cases, and it builds database.
- Tool looks at database to see which parts of the program were executed, and reports test coverage.
- Some open source tools:
 - ✓ Java: https://en.wikipedia.org/wiki/Java_code_coverage_tools.
 - ✓ More languages: <https://stackify.com/code-coverage-tools/>.

Exercise

- Try to input three integers from the keyboard, and then identify the maximum one. If there exist two maximum integers, also point out it.
- First write the source code, and then design test cases to achieve different types of white-box testing coverage.

Black-Box Testing

Black-Box vs. White-Box Techniques for Writing Tests

- White-box (from code)
 - ✓ Code coverage
- **Black-box (from specifications)**
 - ✓ Equivalence partitioning
 - ✓ Boundary value analysis

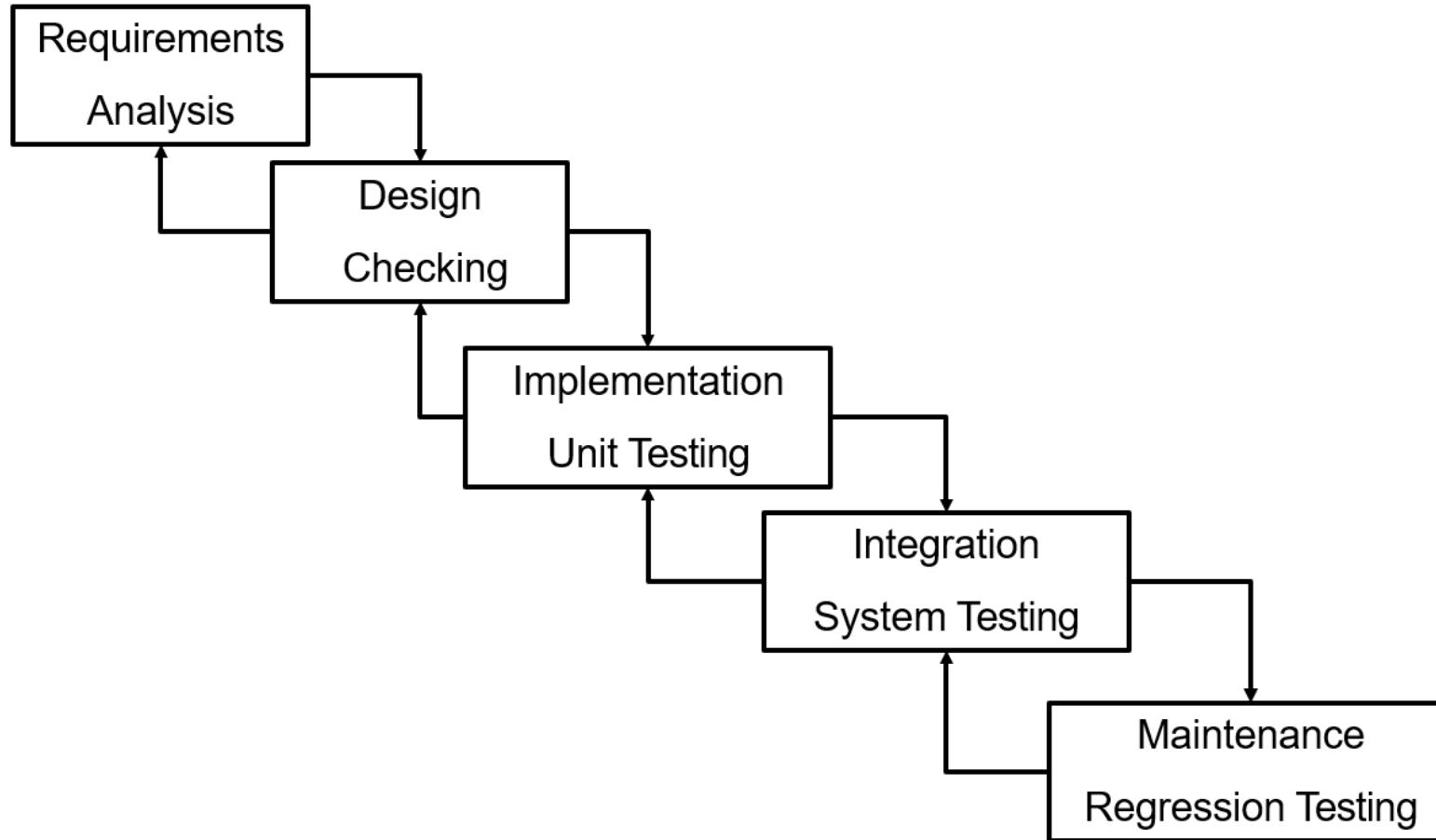
Specification

- Document what the software is supposed to do.
- Should be unambiguous, consistent, clear, simple, ...
 - Example: “If you are senior students and junior students, raise your hand”

Guidelines on Black Box Tests

- Look at **requirements** to generate test cases.
- Test cases need to be **traceable** to a requirement.
- Test cases need to be **repeatable** so anyone on the team can run the exact test case and get the exact same result/sequence of events.
 - ✓ The inputs must be very **specific**.
 - Example: “Students who receive a grade of 70 or higher pass the exam.”
 - Correct test cases: Grade = 80; Grade = 20
 - Incorrect test cases: “input a passing grade” “input a failing grade”
 - ✓ The expected results must be very **specific**. “Pass” or “Fail”.

When to Conduct Black-Box Testing



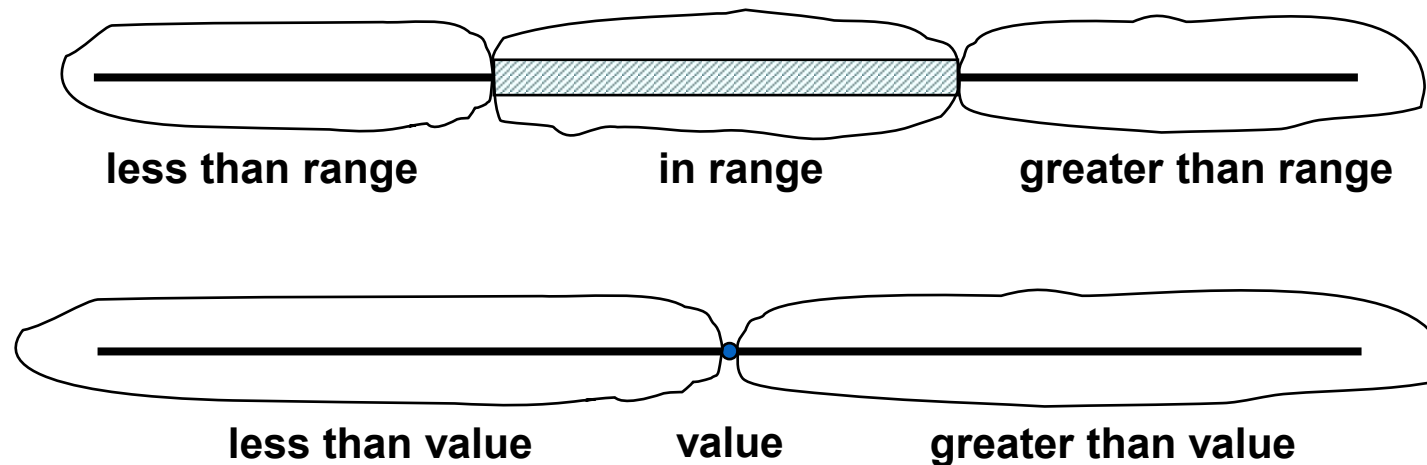
Equivalence Partitioning

Equivalence Partitioning

- Divide input domain into classes (i.e., groups) of equivalent values
 - ✓ Input in the same class should behave similarly in the program.
- Be sure to test a **mid-range value** from each class.
- **Valid** and **invalid** equivalence class
- Example: for tests of “Pass the course” the most important thing is whether the student has a course grade less than 60 points (out of the full 100 points)
 - ✓ Two equivalence partitions: “< 60”; and “>= 60”.
 - ✓ Test input values clearly in the two partitions: 30 and 80.

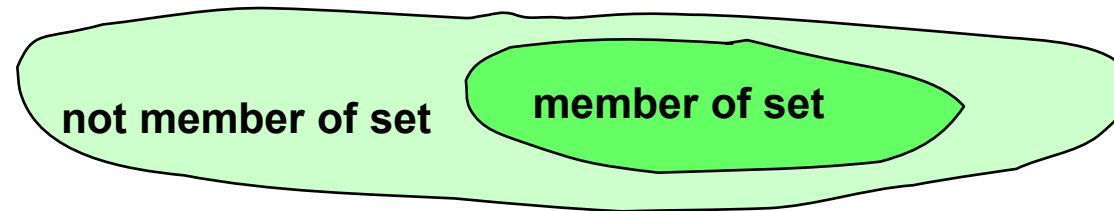
How to Determine Equivalence Classes?

- Case 1: When the input conditions specify the range of values or the number of values, one valid equivalence class and two invalid equivalence classes can be established.

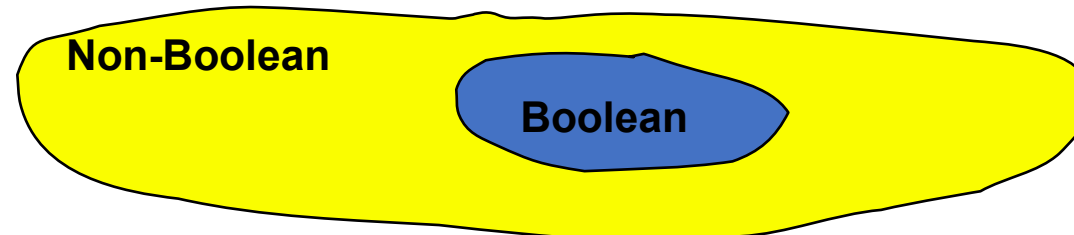


How to Determine Equivalence Classes?

- Case 2: “Must be in a set”



- Case 3: For Boolean inputs



How to Determine Equivalence Classes?

- Case 4: When a set of values (assuming n) of input data is specified, and the program is to process each input value separately, n valid equivalence classes and one invalid equivalence class can be established.
- Case 5: In the case of stipulating the rules that the input data must comply with, one valid equivalence class (conforming to the rule) and several invalid equivalence classes (violating the rule from different angles) can be established.

Construct Test Cases via Equivalence Partitioning

- 1) Establish an equivalence class table to list all equivalence classes divided out, such as **Input Condition (or Input Rule) / Valid Equivalence Class / Invalid Equivalence Class**;
- 2) Specify a unique number for each equivalence class;
- 3) Design a new test case to cover as many effective equivalence classes as possible;
- 4) Repeat 3), and finally make all valid equivalence classes covered by test cases;
- 5) Design a new test case to cover only an invalid equivalence class.
- 6) Repeat 5) so that all invalid equivalence classes are covered.

Equivalence Partitioning: Example 1

- The mailbox name requires 6 to 18 characters. Letters, numbers, underscores can be used, and it must start with a letter. How to use equivalence partitioning to design test cases?

Equivalence Partitioning: Example 1

- The mailbox name requires 6 to 18 characters. Letters, numbers, underscores can be used, and it must start with a letter. How to use equivalence partitioning to design test cases?

Input Condition	Valid Equivalence Class	Invalid Equivalence Class
Mailbox Name		

Equivalence Partitioning: Example 1

- The mailbox name requires 6 to 18 characters. Letters, numbers, underscores can be used, and it must start with a letter. How to use equivalence partitioning to design test cases?

Input Condition	Valid Equivalence Class	Invalid Equivalence Class
Mailbox Name	6-18 characters	
	Letters, numbers, and underscores	
	Start with a letter	

Equivalence Partitioning: Example 1

- The mailbox name requires 6 to 18 characters. Letters, numbers, underscores can be used, and it must start with a letter. How to use equivalence partitioning to design test cases?

Input Condition	Valid Equivalence Class	Invalid Equivalence Class
Mailbox Name	6-18 characters	< 6 characters > 18 characters
	Letters, numbers, and underscores	There exist at least one character that is not a letter, or number or underscore
	Start with a letter	Start with a number or underscore

Equivalence Partitioning: Example 2

- A software system needs the user to input an integer date ranging from “2011-01” (this format) to “2021-12”, i.e., the system can only deal with the date within this range. How to use equivalence partitioning to design test cases?

Equivalence Partitioning: Example 2

- A software system needs the user to input an integer date ranging from “2011-01” (this format) to “2021-12”, i.e., the system can only deal with the date within this range. How to use equivalence partitioning to design test cases?

Input Condition	Valid Equivalence Class	Invalid Equivalence Class
Year		
Connector		
Month		

Equivalence Partitioning: Example 2

- A software system needs the user to input an integer date ranging from “2011-01” (this format) to “2021-12”, i.e., the system can only deal with the date within this range. How to use equivalence partitioning to design test cases?

Input Condition	Valid Equivalence Class	Invalid Equivalence Class
Year	Integer	
	[2011, 2021]	
Connector	“_”	
Month	Integer	
	[01, 12]	

Equivalence Partitioning: Example 2

- A software system needs the user to input an integer date ranging from “2011-01” (this format) to “2021-12”, i.e., the system can only deal with the date within this range. How to use equivalence partitioning to design test cases?

Input Condition	Valid Equivalence Class	Invalid Equivalence Class
Year	Integer	Non-integer
	[2011, 2021]	<2011; > 2021
Connector	“_”	Not “-”
Month	Integer	Non-integer
	[01, 12]	< 01; > 12

Equivalence Partitioning: Example 3

- Program requirements: Input three integers a , b , and c as the length of the three sides of the triangle, and determine the type of triangle formed by the program; when the triangle is a general triangle, an isosceles triangle or an equilateral triangle, deal with them separately.

Equivalence Partitioning: Example 3

- Triangle type problem:

Input Condition		Valid Equivalence Class	Invalid Equivalence Class
Input Domain	Input 3 integers		
Output Domain	General Triangle		
	Isosceles Triangle		
	Equilateral Triangle		

Equivalence Partitioning: Example 3

- Triangle type problem:

Input Condition		Valid Equivalence Class	Invalid Equivalence Class
Input Domain	Input 3 integers	Integer	
		3 numbers	
		Positives	
Output Domain	General Triangle	$a + b > c$ $a + c > b$ $b + c > a$	
	Isosceles Triangle	$a = b \neq c$ $a = c \neq b$ $b = c \neq a$	
	Equilateral Triangle	$a = b = c$	

Equivalence Partitioning: Example 3

- Triangle type problem:

Input Condition		Valid Equivalence Class	Invalid Equivalence Class
Input Domain	Input 3 integers	Integer	Non-integer? How many?
		3 numbers	
		Positives	
Output Domain	General Triangle	$a + b > c$ $a + c > b$ $b + c > a$	
	Isosceles Triangle	$a = b \neq c$ $a = c \neq b$ $b = c \neq a$	
	Equilateral Triangle	$a = b = c$	

Equivalence Partitioning: Example 3

- Triangle type problem:

Input Condition		Valid Equivalence Class	Invalid Equivalence Class
Input Domain	Input 3 integers	Integer	One edge is not an integer (3 cases) Two edges are not integers (3 cases) Three edges are not integers (1 case)
		3 numbers	
		Positives	
Output Domain	General Triangle	$a + b > c$ $a + c > b$ $b + c > a$	
	Isosceles Triangle	$a = b \neq c$ $a = c \neq b$ $b = c \neq a$	
	Equilateral Triangle	$a = b = c$	

Equivalence Partitioning: Example 3

- Triangle type problem:

Input Condition		Valid Equivalence Class	Invalid Equivalence Class
Input Domain	Input 3 integers	Integer	There exists at least one non-integer
		3 numbers	
		Positives	
Output Domain	General Triangle	$a + b > c$ $a + c > b$ $b + c > a$	
	Isosceles Triangle	$a = b \neq c$ $a = c \neq b$ $b = c \neq a$	
	Equilateral Triangle	$a = b = c$	

Equivalence Partitioning: Example 3

- Triangle type problem:

Input Condition		Valid Equivalence Class	Invalid Equivalence Class
Input Domain	Input 3 integers	Integer	There exists at least one non-integer
		3 numbers	?
		Positives	?
Output Domain	General Triangle	$a + b > c$ $a + c > b$ $b + c > a$	?
	Isosceles Triangle	$a = b \neq c$ $a = c \neq b$ $b = c \neq a$	?
	Equilateral Triangle	$a = b = c$	?

Equivalence Partitioning: Example 3

Input Condition		Valid Equivalence Class	Invalid Equivalence Class
Input Domain	Input 3 integers	Integer	There exists at least one non-integer
		3 numbers	Input $k \neq 3$ integers
		Positives	There exists at least non-positive
Output Domain	General Triangle	$a + b > c$	$a + b \leq c$
		$a + c > b$	$a + c \leq b$
		$b + c > a$	$b + c \leq a$
	Isosceles Triangle	$a = b \neq c$	$a \neq b \neq c$
		$a = c \neq b$	
		$b = c \neq a$	
	Equilateral Triangle	$a = b = c$	$a \neq b$
			$b \neq c$
			$a \neq c$

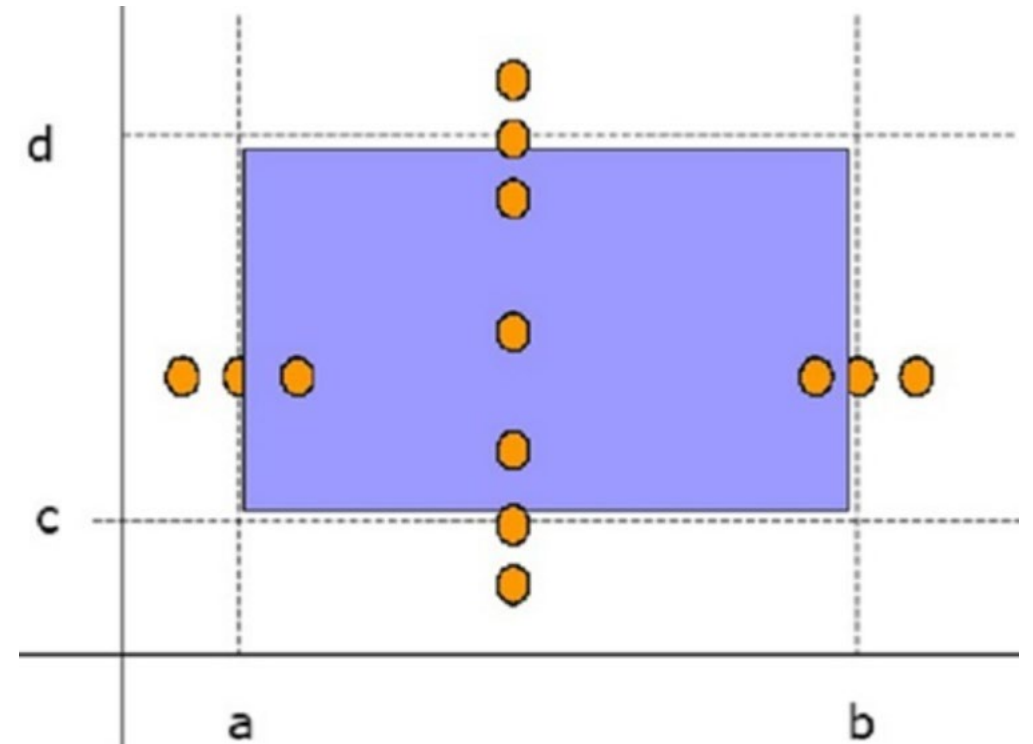
Boundary Value Analysis

Boundary Value Analysis

- Pick test values at the boundary of an equivalence class because faults tend to occur at the boundaries of a domain, not its center.
- Range input, a to b , test with a , b , $a-1$, $a+1$, $b-1$, $b+1$ if integer range; otherwise, slightly less than a and slightly more than b .
- If you can only have a certain quantity (q) of something, try to create $q-1$, q , $q+1$.

Boundary Value Analysis

- Test for **robustness** by going beyond the boundary, including the center of the domain.
- How many test cases are required?
 - The answer is $(6n + 1)$.



Boundary Value Analysis

- Boundary value analysis is combined with equivalence partitioning together, because in general **boundary value analysis should be performed on each equivalence class.**
- In other words, equivalence partitioning is adopted to partitioning the domain into different equivalence classes, and then boundary value analysis is applied to each equivalence class.

Boundary Value Analysis: Example 1

- Suppose three integers x , y , and z as the inputs of the system, and x , y , and z are within $[1, 200]$.

Boundary Value Analysis: Example 1

- Suppose three integers x , y , and z as the inputs of the system, and x , y , and z are within $[1, 200]$.
- Valid equivalence classes? Invalid equivalence classes?

Boundary Value Analysis: Example 1

- Suppose three integers x, y, and z as the inputs of the system, and x, y, and z are within [1, 200].
- Valid equivalence classes? Invalid equivalence classes?

Input Condition	Valid Equivalence Class	Invalid Equivalence Class
Input 3 integers x, y, and z	Integer	
	3 numbers	
	[1, 200]	

Boundary Value Analysis: Example 1

- Suppose three integers x, y, and z as the inputs of the system, and x, y, and z are within [1, 200].
- Valid equivalence classes? Invalid equivalence classes?

Input Condition	Valid Equivalence Class	Invalid Equivalence Class
Input 3 integers x, y, and z	Integer	There exist at least one non-integer
	3 numbers	Input k != 3 integers
	[1, 200]	< 1; > 200

Boundary Value Analysis: Example 1

- Suppose three integers x, y, and z as the inputs of the system, and x, y, and z are within [1, 200].
- Valid equivalence classes? Invalid equivalence classes?
- How to design test cases using boundary value analysis?

Input Condition	Valid Equivalence Class	Invalid Equivalence Class
Input 3 integers x, y, and z	Integer	There exist at least one non-integer
	3 numbers	Input k != 3 integers
	[1, 200]	< 1; > 200

Boundary Value Analysis: Example 1

- Based on valid equivalence class, adopt boundary value analysis;
- Similarly, based on invalid equivalence class, adopt boundary value analysis. For example, “There exist at least one non-integer” can be “The number of non-integers is in $[1, 3]$ ”; while “input $\neq 3$ integers” can be “ k is in $[0, 3)$ ” and “ $k \geq 4$ ”.

Input Condition	Valid Equivalence Class	Invalid Equivalence Class
Input 3 integers x, y, and z	Integer	There exist at least one non-integer
	3 numbers	Input $k \neq 3$ integers
	$[1, 200]$	< 1 ; > 200

Boundary Value Analysis: Example 2

- Suppose the users are allowed to input a positive integer x with 6 digits.

Boundary Value Analysis: Example 2

- Suppose the users are allowed to input a positive integer x with 6 digits.
- Valid equivalence classes? Invalid equivalence classes?
- How to design test cases using boundary value analysis?

Boundary Value Analysis: Example 2

- Suppose the users are allowed to input a positive integer x with 6 digits.
- Valid equivalence classes? Invalid equivalence classes?
- How to design test cases using boundary value analysis?

Input Condition	Valid Equivalence Class	Invalid Equivalence Class
x	Integer	
	Positive	
	6 digits	

Boundary Value Analysis: Example 2

- Suppose the users are allowed to input a positive integer x with 6 digits.
- Valid equivalence classes? Invalid equivalence classes?
- How to design test cases using boundary value analysis?

Input Condition	Valid Equivalence Class	Invalid Equivalence Class
x	Integer	Non-integer
	Positive	Non-positive
	6 digits	Not 6-digits

Boundary Value Analysis: Example 2

- Based on valid equivalence class, adopt boundary value analysis;
- Similarly, based on invalid equivalence class, adopt boundary value analysis. For example, “Non-integer” can be “The number of non-integer digits is in [1, 6]”; while “Not 6 digits” can be “less than 6 digits” and “more than 6 digits”.

Input Condition	Valid Equivalence Class	Invalid Equivalence Class
x	Integer	Non-integer
	Positive	Non-positive
	6 digits	Not 6-digits

Boundary Value Analysis: Example 3

- If an integer x with 6 digits is to represent a data with format “yyyymm”, and yyyy is in [2000, 2021], mm is in [01, 12], try to use equivalence partitioning and boundary value analysis to design test cases?

Boundary Value Analysis: Example 3

- If an integer x with 6 digits is to represent a data with format “yyyymm”, and yyyy is in [2000, 2021], mm is in [01, 12], try to use equivalence partitioning and boundary value analysis to design test cases?
- Based on the valid equivalence class (an integer x with 6 digits), consider “yyyy” and “mm” to re-design valid and invalid equivalence classes.

Input Condition	Valid Equivalence Class	Invalid Equivalence Class
x	Integer	Non-integer
	Positive	Non-positive
	6 digits	Not 6-digits

Boundary Value Analysis: Example 4

8. Attendee Privileges

Select privileges that you want **attendees** to have when meeting begins:

Privileges:

- ☐ Save
- ☐ Print
- ☒ Annotate
- ☒ View participants list
- ☐ Control next or previous images
- ☐ View thumbnails
- ☒ Control applications, Web browser or desktop remotely
- ☐ View any document
- ☒ View any page
- ☐ Contact operator privately

Participant in private chat with:

- ☐ Host
- ☐ Presenter
- ☐ All attendees

- 1 Required Information
- 2 Date & Time
- 3 Teleconference
- 4 Invite Attendees
- 5 Registration
- 6 Agenda & Welcome
- 7 Meeting Options
- 8 Attendee Privileges**
- 9 Review

Cancel

7

Back

Next

9

START

Unit Testing

What is a Unit Test?

- A unit test is a small program with assertions.

```
@Test
void testHashSetAddTwoElems()
{
    HashSet set = new HashSet();
    set.add(3);
    set.add(14);
    assertEquals(set.size(), 2);
}
```

- Many developers write such unit tests by hand, involving
 - ✓ determining a meaningful sequence of method calls,
 - ✓ selecting exemplary argument values (the test inputs),
 - ✓ stating assertions (test oracles)

Test Generalization

- Test generalization is to transform conventional tests to parameterized tests.
- Consider an example of conventional tests (CTs) for stack.

Example Conventional Tests (CTs) for Stack

```
@Test
void CT1() {
    int elem = 1;
    Stack<Integer> stk = new Stack<Integer>();
    stk.push(elem);
    assertEquals(1, stk.size());
}
```

```
@Test
void CT2() {
    int elem = 30;
    Stack<Integer> stk = new Stack<Integer>();
    stk.push(elem);
    assertEquals(1, stk.size());
}
```

```
@Test
void CT3() {
    int elem1 = 1;
    int elem2 = 30;
    Stack<Integer> stk = new Stack<Integer>();
    stk.push(elem1);
    stk.push(elem2);
    assertEquals(2, stk.size());
}
```

- CT1 and CT2 exercise *push* with different test data
- CT3 exercises *push* when stack is not empty

Two main issues with CTs:

- Fault-detection capability issue: undetected defect where things go wrong when passing a negative value to *push*
- Redundant test issue: CT2 is redundant with respect to CT1

Test Generalization

- Step 1: Parameterize
- Step 2: Generalize Test Oracle
- Step 3: Add Assumptions
- Step 4: Cross-test generalize

Step 1: Parameterize

- Promote all **primitive** values as arguments: *elem* as a parameter of type `int`.
- Promote **non-primitive** objects such as receiver objects as arguments: *stk* as a parameter of type `Stack<Integer>`.

```
@Test
void CT1() {
    int elem = 1;
    Stack<Integer> stk = new Stack<Integer>();
    stk.push(elem);
    assertEquals(1, stk.size());
}
```



```
@ParameterizedTest
@ValueSource(...)
void PT1(int elem, Stack<Integer> stk) {
    stk.push(elem);
    assertEquals(1, stk.size());
}
```

```
@Test
void CT2() {
    int elem = 30;
    Stack<Integer> stk = new Stack<Integer>();
    stk.push(elem);
    assertEquals(1, stk.size());
}
```



```
@ParameterizedTest
@ValueSource(...)
void PT2(int elem, Stack<Integer> stk) {
    stk.push(elem);
    assertEquals(1, stk.size());
}
```

Step 1: Parameterize

- Promote all **primitive** values as arguments: *elem* as a parameter of type `int`.
- Promote **non-primitive** objects such as receiver objects as arguments: *stk* as a parameter of type `Stack<Integer>`.

```
@Test
void CT3() {
    int elem1 = 1;
    int elem2 = 30;
    Stack<Integer> stk = new Stack<Integer>();
    stk.push(elem1);
    stk.push(elem2);
    assertEquals(2, stk.size());
}
```



```
@ParameterizedTest
@ValueSource(...)
void PT3(int elem1, int elem2, Stack<Integer> stk) {
    stk.push(elem1);
    stk.push(elem2);
    assertEquals(2, stk.size());
}
```

Step 2 – Generalize Test Oracle

- Replace **constant** values in assertions with some generalized expressions, e.g., 1 and 2

```
@ParameterizedTest
@ValueSource(...)
void PT1(int elem, Stack<Integer> stk) {
    stk.push(elem);
    assertEquals(1, stk.size());
}
```



```
@ParameterizedTest
@ValueSource(...)
void PT1(int elem, Stack<Integer> stk) {
    int oldSize = stk.size();
    stk.push(elem);
    assertEquals(oldSize + 1, stk.size());
}
```

```
@ParameterizedTest
@ValueSource(...)
void PT3(int elem1, int elem2, Stack<Integer> stk) {
    stk.push(elem1);
    stk.push(elem2);
    assertEquals(2, stk.size());
}
```



```
@ParameterizedTest
@ValueSource(...)
void PT3(int elem1, int elem2, Stack<Integer> stk) {
    int oldSize = stk.size();
    stk.push(elem1);
    stk.push(elem2);
    assertEquals(oldSize + 2, stk.size());
}
```

Step 3 – Add Assumptions

- Add assumptions to prevent **illegal** values for the parameters or enable the assertions to be **valid**, e.g., `AddassumeTrue(stk!=null)`, i.e., the generated value should not be null.

```
@ParameterizedTest
@ValueSource(...)
void PT1(int elem, Stack<Integer> stk) {
    int oldSize = stk.size();
    stk.push(elem);
    assertEquals(oldSize + 1, stk.size());
}
```



```
@ParameterizedTest
@ValueSource(...)
void PT1(int elem, Stack<Integer> stk) {
    assumeTrue(stk != null);
    int oldSize = stk.size();
    stk.push(elem);
    assertEquals(oldSize + 1, stk.size());
}
```

```
@ParameterizedTest
@ValueSource(...)
void PT3(int elem1, int elem2, Stack<Integer> stk) {
    int oldSize = stk.size();
    stk.push(elem1);
    stk.push(elem2);
    assertEquals(oldSize + 2, stk.size());
}
```



```
@ParameterizedTest
@ValueSource(...)
void PT3(int elem1, int elem2, Stack<Integer> stk) {
    assumeTrue(stk != null);
    int oldSize = stk.size();
    stk.push(elem1);
    stk.push(elem2);
    assertEquals(oldSize + 2, stk.size());
}
```

Step 4 – Cross-test Generalize

- Generalize further across **multiple tests**, e.g., further generalize PT1 and PT3.

```
@ParameterizedTest
@ValueSource(...)
void PT1(int elem, Stack<Integer> stk) {
    assertTrue(stk != null);
    int oldSize = stk.size();
    stk.push(elem);
    assertEquals(oldSize + 1, stk.size());
}
```

```
@ParameterizedTest
@ValueSource(...)
void PT3(int elem1, int elem2, Stack<Integer> stk) {
    assertTrue(stk != null);
    int oldSize = stk.size();
    stk.push(elem1);
    stk.push(elem2);
    assertEquals(oldSize + 2, stk.size());
}
```



```
@ParameterizedTest
@ValueSource(...)
void PT(int[] elem) {
    assertTrue(elem != null);
    Stack<Integer> stk = new Stack<Integer>();
    for(int i=0; i< elem.length; i++)
        stk.push(elem[i]);
    assertEquals(elem.length, stk.count());
}
```


Unit Testing: Benefits

- Design and specification
 - ✓ by example
- Code coverage and regression testing
 - ✓ confidence in correctness
 - ✓ preserving behavior
- Short feedback loop
 - ✓ unit tests exercise little code
 - ✓ failures are easy to debug
- Documentation

Tests as Specifications

- Tests show how to use the system
- Tests need to be readable
 - ✓ Need comments that describe their purpose or need good names
 - ✓ Keep short, and delete duplicate or redundant tests

```
@Test
void testHashSetAddTwoElems()
{
    HashSet set = new HashSet();
    set.add(3);
    set.add(14);
    assertEquals(set.size(), 2);
}
```

Unit Testing: Measuring Quality

- Coverage: Are all parts of the program exercised?
 - ✓ statements
 - ✓ basic blocks
 - ✓ explicit/implicit branches
 - ✓ ...
- Assertions: Does the program do the right thing?
 - ✓ test oracle
- Experience:
 - ✓ Just high coverage or large number of assertions is no good quality indicator.
 - ✓ Only both together are!

xUnit Testing Frameworks

- Developers' testing frameworks/tools such as JUnit
 - ✓ Each code unit requiring several tests
- Automated testing!
 - ✓ Unit testing, but also integration testing and functional testing
 - ✓ Regression testing

Software Debugging

Debugging vs. Testing

- What is the difference?
- How would you describe it in terms of failures and faults?

Recall: Mistake, Fault, Error, Failure

What does *Bug* mean in "*Bug Report*"?

Developer makes a **mistake**.

Fault (**defect, bug**) introduced in the software.

Fault remains undetected during testing

i.e., run **test inputs** & check **outputs**

The program **fails** (based on **test oracles**) during execution

i.e., it behaves unexpectedly

Error: difference between (1) computed, observed, or measured value/condition & (2) true, specified, or theoretically correct value/condition

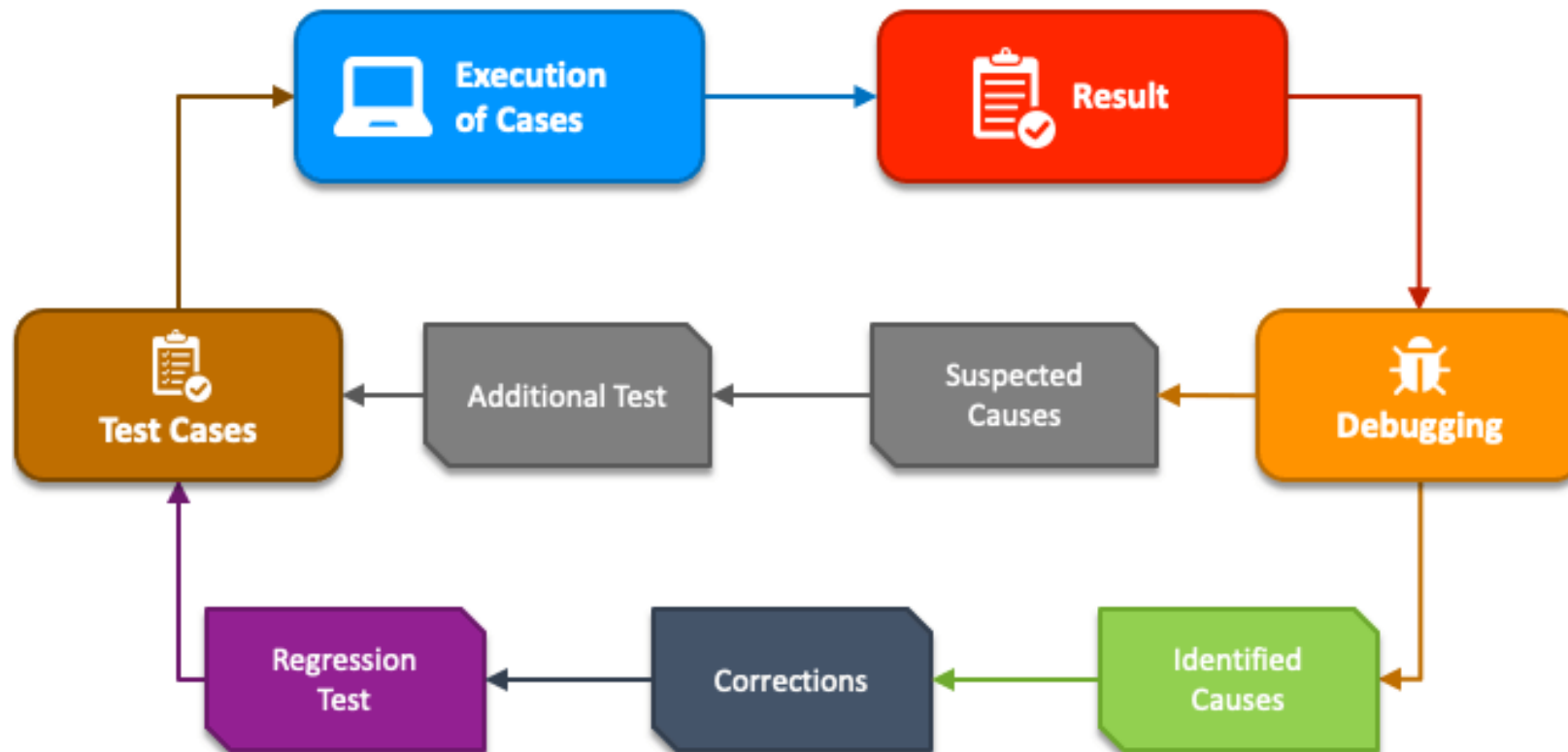
From Testing to Debugging

- Purpose of testing is to find **failure**
 - ✓ Failure: symptom - situation that gives wrong result
 - ✓ Producing tests that fail
- Purpose of debugging (given a failure) is to locate and fix the **fault**
 - ✓ PFault: cause – mistake made in the code
 - ✓ Producing fixing changes to the code

(Program) Debugging

- **Locate** and **fix** fault (root cause) for causing a program not to behave as expected
 - Note that root cause can be elsewhere, e.g., program configuration, data, hardware
- Need to **re-test** the program after debugging to make sure
 - Fault gets **fully** fixed
 - **No** new faults are introduced during the fix

Debugging Process



Debugging Process – TRAFFIC

- **T**rack the problem
 - Enter a problem (aka. bug) report into a problem tracker (e.g., Bugzilla)
- **R**eproduce
- **A**utomate
 - Write system/unit tests to reproduce automatically
- **F**ind Origins
 - Trace from known infection point (failure point) backward along the infection chain
- **F**ocus
 - Search for locations possibly wrong (being faulty before) or explicitly wrong (**assertion** violation)
- **I**solate
 - Systematically narrow down failure causes using **observation** and **experiments**
- **C**orrect

Testing-Based Fault Localization

Testing-Based Fault Localization

- Fault localization is the process of identifying the exact location or cause of faults, such as bugs or errors within software.
- Fault localization techniques can be broadly classified into different categories, while the most popular one is testing-based fault localization.

Testing-Based Fault Localization

- Input:
 - ✓ The source code of the software system;
 - ✓ A set of tests, at least one of which fails.
- Output: A list of potentially faulty program elements, sorted by probability of failure.
- Program elements can be defined at different levels:
 - ✓ Expressions
 - ✓ Statements
 - ✓ Methods
 - ✓ Classes
 - ✓ Files...

Testing-Based Fault Localization

- Different types of testing-based fault localization:
 - ✓ Test-coverage-based fault localization;
 - Spectrum-based fault localization
 - ✓ State-coverage-based fault localization;
 - ✓ Mutation-based fault localization;

Spectrum-Based Fault Localization

Spectrum-Based Fault Localization

- The most widely used automated error localization method is simple in form and effective.
- Program Spectrum:
 - ✓ First invented by Tom Reps of the University of Wisconsin in 1997 while dealing with the Y2K problem
 - ✓ It refers to statistics during program execution.
- Spectrum-based fault localization:
 - ✓ James Jone, Mary Jean Harrold, et al. of Georgia Tech in 2002 generalized Tom Reps' method into a universal debugging method.
 - ✓ The main spectrum information used is test coverage information.

Spectrum-Based Fault Localization

- The most widely used automated error localization method is simple in form and effective.
- Program Spectrum:
 - ✓ First invented by Tom Reps of the University of Wisconsin in 1997 while dealing with the Y2K problem
 - ✓ It refers to statistics during program execution.
- Spectrum-based fault localization:
 - ✓ James Jone, Mary Jean Harrold, et al. of Georgia Tech in 2002 generalized Tom Reps' method into a universal debugging method.
 - ✓ The main spectrum information used is test coverage information.

Spectrum-Based Fault Localization

- Basic Idea:
 - ✓ Program elements executed by failed test cases are more likely to have errors.
 - ✓ Program elements executed by successful test cases are more likely to be error-free.
- Program elements can be defined at different granularities:
 - ✓ Basic Block
 - ✓ Method
 - ✓ Class
 - ✓ File
 - ✓ Can it be a statement or an expression?

Spectrum-Based Fault Localization

		T15	T16	T17	T18
1	<pre> int count; int n; Ele *proc; List *src_queue, *dest_queue; if (prio >= MAXPRIO) /*maxprio=3*/ </pre>	●	●	●	●
2	<pre> {return;} </pre>	●			
3	<pre> src_queue = prio_queue[prio]; dest_queue = prio_queue[prio+1]; count = src_queue->mem_count; if (count > 1) /* Bug*//* supposed : count>0*/ { </pre>		●	●	●
4	<pre> n = (int) (count*ratio + 1); proc = find_nth(src_queue, n); if (proc) { </pre>		●	●	
5	<pre> src_queue = del_ele(src_queue, proc); proc->priority = prio; dest_queue = append_ele(dest_queue, proc); } </pre>		●	●	
Pass/Fail of Test Case Execution :		Pass	Pass	Pass	Fail

Calculate the Degree of Suspicion of Program Elements

- a_{ef} : The number of failed tests for executing statement a ;
- a_{nf} : The number of failed tests for not executing statement a ;
- a_{ep} : The number of passed tests for executing statement a ;
- a_{np} : The number of passed tests for not executing statement a .

Tarantula:
$$\frac{a_{ef}}{a_{ef} + a_{nf}} / \left(\frac{a_{ef}}{a_{ef} + a_{nf}} + \frac{a_{ep}}{a_{ep} + a_{np}} \right)$$

Jaccard:
$$\frac{a_{ef}}{a_{ef} + a_{nf} + a_{ep}}$$

Ochiai:
$$\frac{a_{ef}}{\sqrt{(a_{ef} + a_{nf})(a_{ef} + a_{ep})}}$$

D*:
$$\frac{a_{ef}^*}{a_{nf} + a_{ep}}, \text{ * is equal to 2 or 3}$$

Which Formula is the Best?

- Experimental Verification:
 - ✓ Experimental results are inconsistent across different objects.
 - ✓ Early experiments suggested Ochiai was the best, while the D* paper argued that D* was the best.
 - ✓ Recent research on real-world Java defects suggests no statistically significant differences between different formulas: The top-5 statements achieve an average accuracy of 18% in locating key points, while the top-10 achieve 27%.
- Theoretical Research:
 - ✓ Xie et al. from Wuhan University theoretically proved that no single best formula exists.
 - ✓ David Lo et al. from Singapore Management University conducted experiments that yielded conclusions inconsistent with Xie's findings.

Other Possible Coverage Information

- Previously, we considered direct coverage of program elements, while other coverage types can also be considered:
 - ✓ Branch coverage: Coverage for each selection in a branch statement.
 - ✓ Data flow coverage: Coverage for each variable definition-use pair.
- Locating a specific coverage unit indicates that the statement affecting that unit may be incorrect.
 - ✓ Branch coverage: The corresponding branch may be incorrect, or the condition of the branch statement may be incorrect.
 - ✓ Data flow coverage: The corresponding definition-use pair may be incorrect.

Mutation-Based Fault Localization

Mutation-Based Fault Localization

- Mutation: Any random modification to a program.
- Mutation Analysis: Analyzing whether the original tests have been used in the mutated program.
- Mutation analysis is widely used in testing to evaluate the quality of a test set.
 - ✓ If any test in a test set fails to execute on a mutated program, the mutant is said to have been killed by that test.
 - ✓ A test set that kills as many mutants as possible is considered a good test set.

Mutation-Based Fault Localization

- C: Milu
- Java
 - ✓ MuJava: Basic mutation testing tool with relatively complete support for mutation operators;
 - ✓ Javalanche: Supports acceleration of Mutation;
 - ✓ SchemataMajor: Supports acceleration of pre-filtered test execution, but supports fewer mutation operators;
 - ✓ PIT: Commercial tool, with the most complete features and fastest speed.

Mutation-Based Fault Localization

- Assumption 1: When mutating a faulty statement,
 - ✓ The probability of a failed test case output changing is greater than the probability of a passed test case output changing.
- Assumption 2: When mutating a correct statement,
 - ✓ The probability of a failed test case output changing is less than the probability of a passed test case output changing.
- Assumption 3: The probability of a failed test becoming a passed test
 - ✓ When mutating a faulty statement is greater than mutating a passed test case output.
- Assumption 4: The probability of a passed test becoming a failed test
 - ✓ When mutating a correct statement is greater than mutating a failed test case output.

Metallaxis

- In 2012, Professor Yves Le Traon and colleagues from the University of Luxembourg proposed the following formula:
- Variant Suspicion Formula: $\frac{m_f}{\sqrt{F * (m_f + m_p)}}$
 - ✓ m : Variant,
 - ✓ m_f : Number of failed tests where the output changed,
 - ✓ m_p : Number of passed tests where the output changed,
 - ✓ F : Original total number of failed tests.
- The suspicion of a program element is the suspicion of the highest variant on that element.

MUSE

- Moonzoo Kim of the Korea Advanced Institute of Science and Technology (KAIST) and Shin Yoo of the University College London (UCL) proposed the following formula in 2014:
- Variant Suspicion Formula: $m_{f2p} - m_{p2f} \frac{\sigma_m m_{f2p}}{\sigma_m m_{p2f}}$
 - ✓ m : Variant,
 - ✓ m_{f2p} : The number of tests on m that changed from failure to success,
 - ✓ m_{p2f} : The number of tests on m that changed from success to failure.
- The suspiciousness of a program element is the average suspiciousness of its variants.

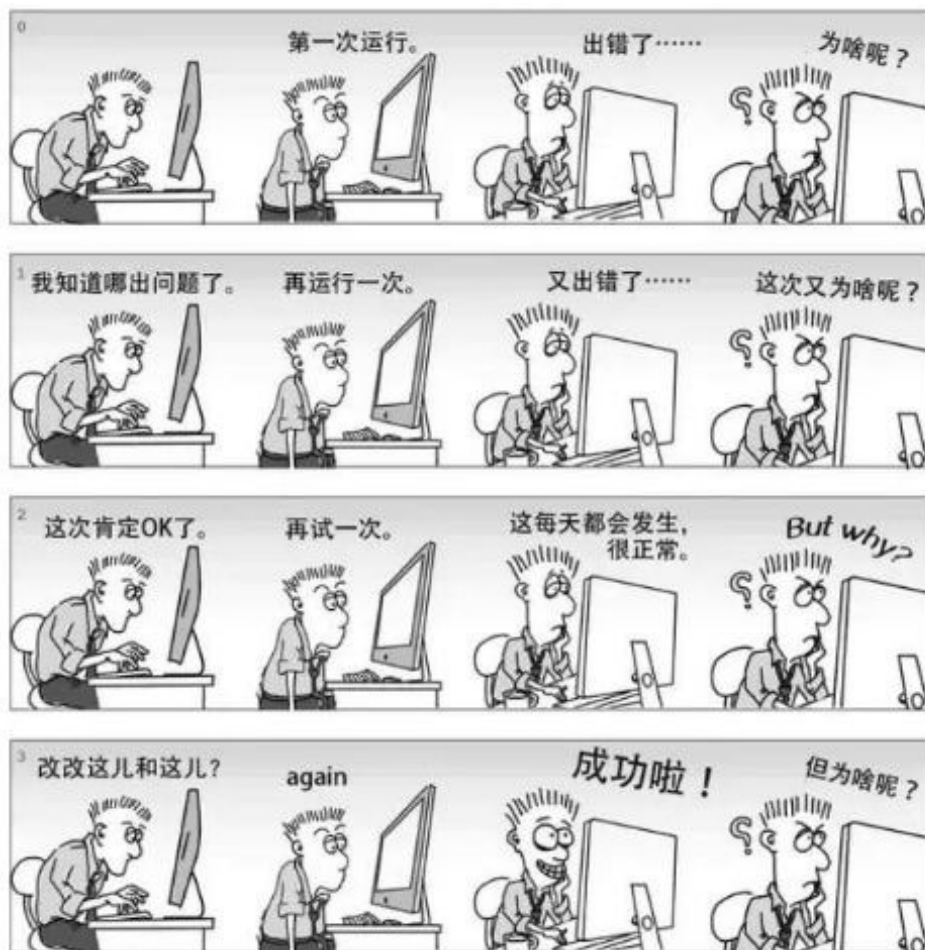
SBFL vs. MBFL

- In spectrum-based fault localization, different tests that cover the same statement will have the same effect on the result.
- However, different tests are affected differently by the same statement.
 - ✓ Different tests have different probabilities of triggering faults.
 - ✓ Different tests have different probabilities of propagating faults.
 - ✓ Different tests have different probabilities of catching faults.
- Mutation-based fault localization actually relies on mutation to capture the relationship between tests and statements.

Program Repair (or Defect Repair)

The Life of a Programmer

- It's a tragic tale where writing bugs and fixing bugs are intertwined.



How Much Time was Spent Fixing the Bug?

- 35.6% of software maintenance time is spent fixing bugs [1]: Software maintenance costs are generally considered to account for 90% of software costs.
- Developers spend about half of their total development time on fixing bugs [2].
- Development teams may not have enough resources to fix all defects [3].
- Software is released with known defects [4].

[1] B. P. Lientz, E. B. Swanson, and G. E. Tompkins, "Characteristics of application software maintenance," *Commun. ACM*, vol. 21, no. 6, pp. 466–471, 1978

[2] Britton et al. Quantify the time and cost saved using reversible debuggers. Cambridge report, 2013

[3] J. Anvik, L. Hiew, and G. C. Murphy, "Coping with an open bug repository," *eXchange*, 2005, pp. 35–39

[4] B. Liblit, A. Aiken, A. X. Zheng, and M. I. Jordan, "Bug isolation via remote program sampling," in *PLDI*, 2003, pp. 141–154

Automatic Defect Repair

- Input: A program and its correctness constraints, where the program does not satisfy the correctness constraints.
- Output: A patch that makes the program satisfy the constraints.
- The most widely considered correctness constraints in research and practice—testing in software projects.

Roles of Automatic Defect Repair

- Yida Tao, Jindae Kim, Sunghun Kim, Chang Xu: Automatically generated patches as debugging aids: a human study. SIGSOFT FSE 2014: 64-74.
 - ✓ When programmers have high-quality patches to assist them, the accuracy of fixes improves significantly, while the fix time decreases slightly.
 - ✓ Significantly improved fix accuracy => Significantly reduced future fix time.

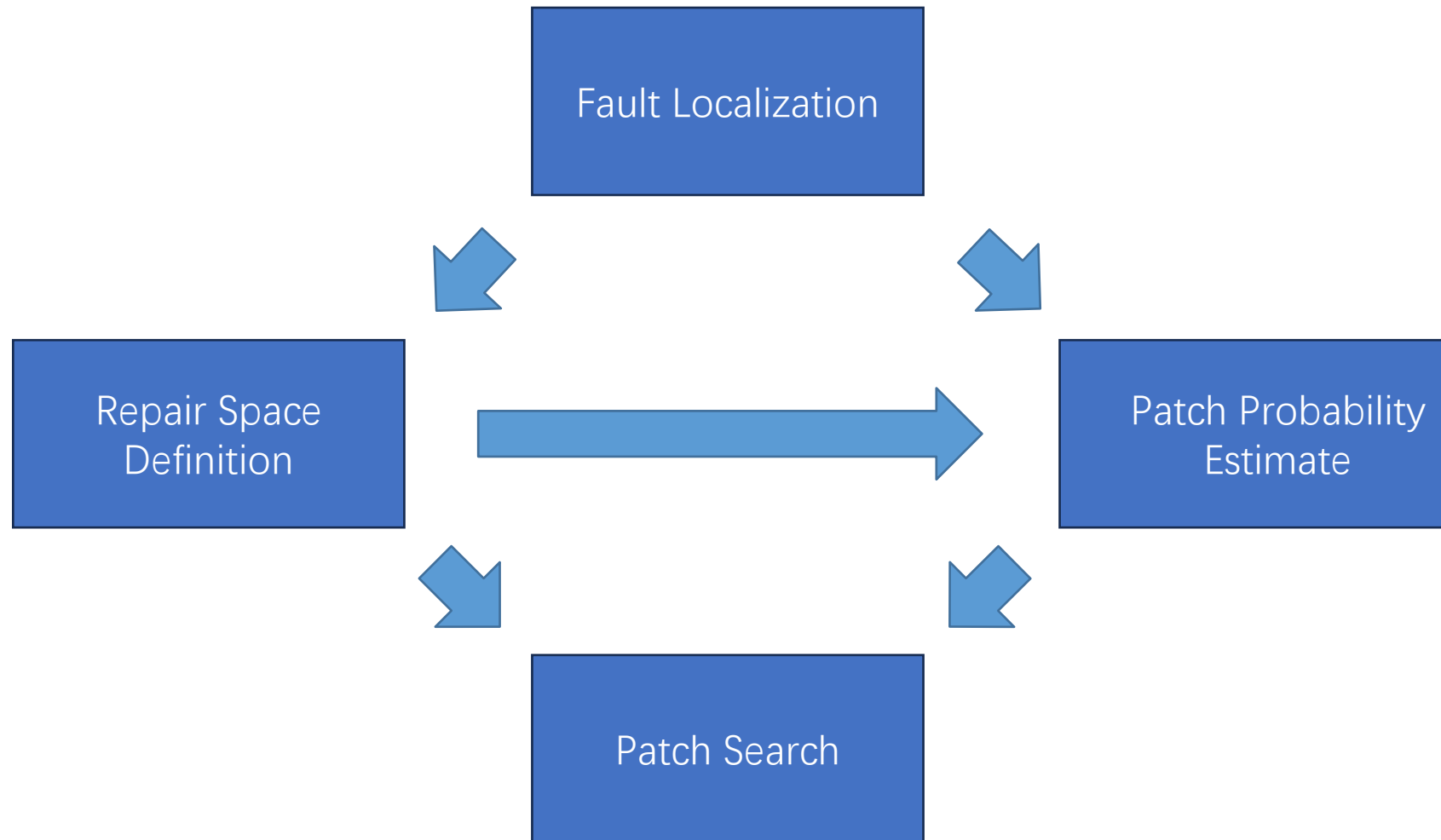
Academic Value of Automatic Defect Repair

- "Automatic program generation is the most fundamental problem in computer science."
 - ✓ Amir Pnueli, Turing Award winner,
 - ✓ On the Synthesis of a Reactive Module*, POPL 1989
- "Program defect repair and automatic program generation are equivalent problems." –
 - ✓ Academician Lin Huimin, Yanqi Lake Conference 2018
- Given a specification and a blank program, if we can repair the defects of the blank program relative to the specification, we have automatically generated a program for that specification.

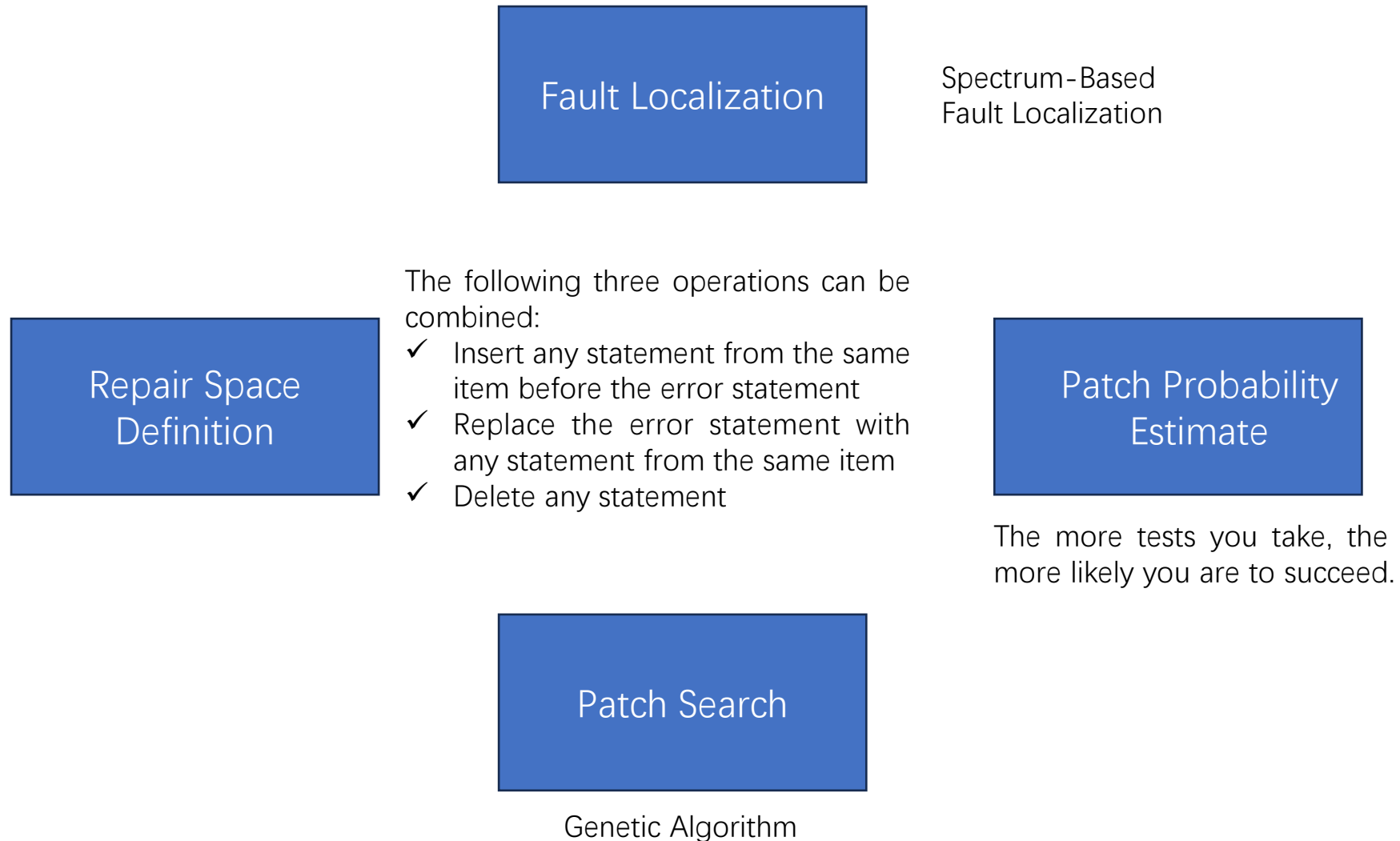
Key Quality Indicators for Defect Repair

- Accuracy: The percentage of correct patches generated, which determines the usability of the technology.
- Recall: The percentage of all defects that can be fixed, which determines the effectiveness of the technology's application.
- Fixing Efficiency: How much time is required to fix each defect, which determines the application scenarios for the technology.

Modern Defect Repair Basic Modules



Typical Defect Repair Technology: GenProg



Typical Defect Repair Technology: ACS

Fault Localization

Spectrum-Based Fault Localization +
Predicate Switching

Repair Space
Definition

Generate conditions like $x \ ? \ v$:

- ✓ x is a variable
- ✓ $?$ is a binary predicate, such as $>$, $\leq$, instanceof
- ✓ v is a constant

Patch Probability
Estimate

- ✓ Ranking the probabilities of variable x using heuristics;
- ✓ ranking the probabilities of $?$ and v using conditional data from GitHub.

Patch Search

Verify in order of probability.

Fault Localization

- Most methods rely on existing fault localization techniques.
- Commonly, spectrum-based fault localization is used.
 - ✓ Program elements executed by failed test cases are more likely to have faults.
 - ✓ Program elements executed by successful test cases are more likely to be error-free.
- StackTrace localization and Predicate Switching techniques are also used for specific defect categories.
- Some methods employ their own localization techniques. AutoFix and JAID locate errors by analyzing invariants.

Definition of Repair Space

- Repair space definition based on program transformation template (PAR):
 - ✓ Swap function variables
 - ✓ Change function names
- Syntax-based repair space definition (SemFix, Angelix, ACS)
 - ✓ Replace erroneous conditional expressions with expressions of the form $x ? V$
- Code comparison-based repair space definition (GenProg, ssFix)
- Statistically identified common operations-based repair space definition (CapGen, SimFix)
- Invariant-based derivation (ClearView, JAID)

Patch Probability Estimate

- Sorting based on fixed rules:
 - ✓ Minimal changes are better [DirectFix, Angelix]
 - ✓ Minimal impact on runtime behavior is better [Qlose]
 - ✓ Small impact on passing tests, large impact on failing tests [Xiong-ICSE18]
- Custom sorting rules [S3]
- Sorting based on statistics/machine learning:
 - ✓ Overall sorting of patches in the patch space [Prophet, Elixir]
 - ✓ Sorting each part of the patch separately and then integrating [ACS, CapGen]
- Integrating the sorting of bug localization:
 - ✓ Using the skepticism of bug localization to correct the likelihood of patch changes [Prophet, Angelix]
 - ✓ Sorting based on the likelihood of invariants [AutoFix, JAID]

Patch Search

- Verify by sorted order [Prophet, Elixir]
- Greedy search by sorted patch parts [ACS]
- Use heuristic search. Genetic algorithm [GenProg]
- Speed up verification
 - ✓ Eliminate compilation redundancy [Mao-ICSME13]
 - ✓ Eliminate runtime inefficiencies [Xiong-ISSTA17]

Specific Types of Defect Repair

- Memory leak fixes
 - Deadlock fixes
 - Competition event fixes
 - Build failure fixes
 - Android crash fixes
 - Job fixes
-
- By designing specific fix spaces and specialized search algorithms, better fix results can usually be achieved.

Defect Repair Community Resources

- Community website: <http://program-repair.org/Defect>
- Repair Overview:
 - ✓ Automatic software repair: a survey. Luca Gazzola, Daniela Micucci, Leonardo Mariani. ICSE 2018: 1219
 - ✓ Automatic Software Repair: A Bibliography. Martin Monperrus. ACM Comput. Surv. 51(1): 17:1-17:24 (2018)

Software Maintenance

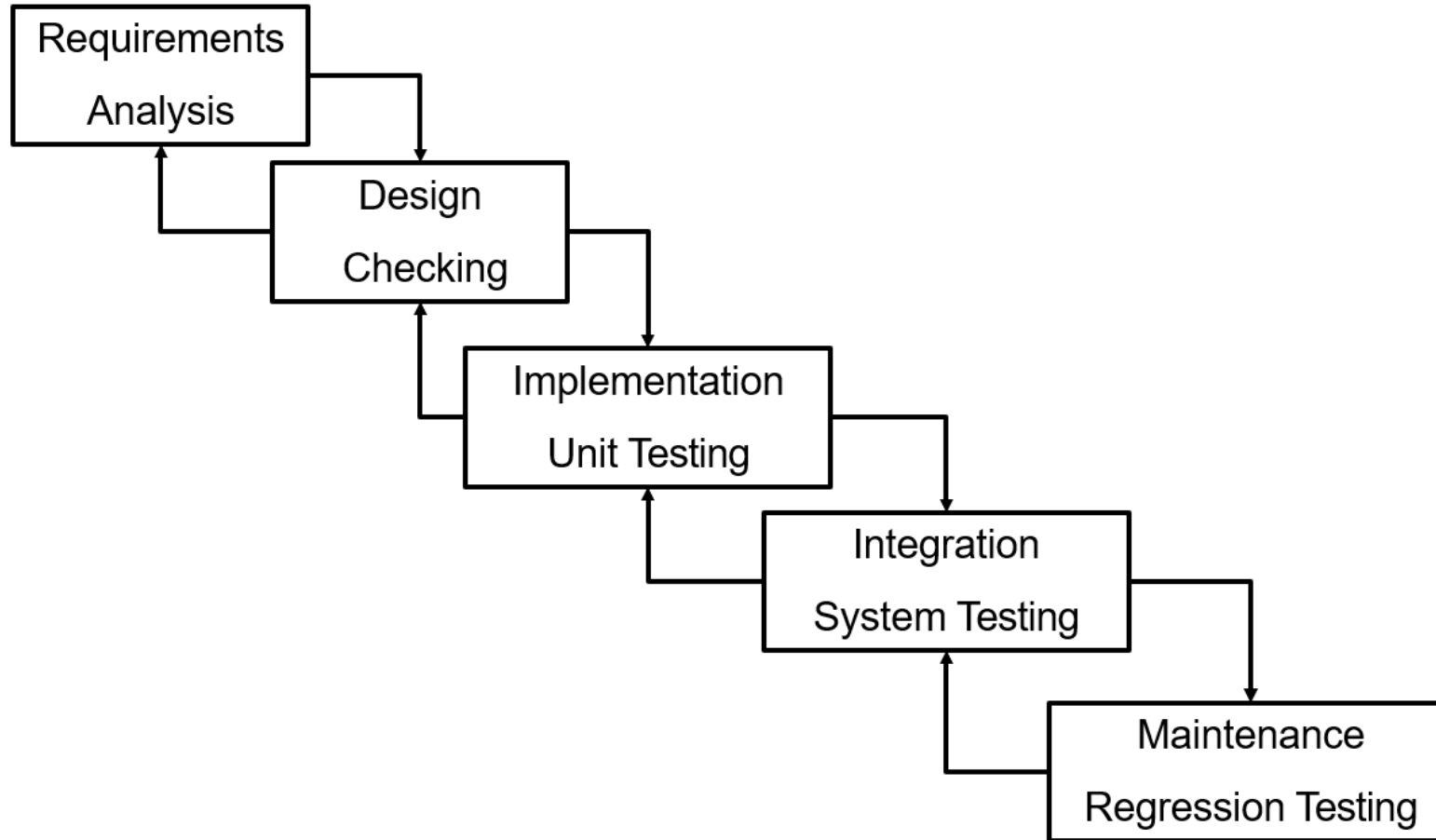
A Fairly Tale

- Once upon a time there was a Good Software Engineer whose Customers knew **exactly** what they wanted.
- The Good Software Engineer worked very hard to design the **Perfect** System that would solve **all** the Customers' problems **now** and **for decades**.
- When the Perfect System was designed, implemented and finally deployed, the Customers were **very happy** indeed.
- The Maintainer of the system had **very little** to do to keep the Perfect System up and running, and the Customers and the Maintainer lived **happily every after**."

Software Maintenance

- Software maintenance is the process of modifying a software product after its delivery to correct faults, improve performance, or adapt it to a changing environment.

Reality: Software Changes



Reality: Software is Getting Complex

(Potvin & Levenberg, 2016)

- Google Codebase (as of January 2015)
 - > 25,000 Google software developers
 - ~ 1 billion files
 - ~ 9 million source files
 - ~ 2 billion lines of source code
 - ~ 35 million commits spanning 18 years
 - ~15 million lines of source code being changed weekly in 2014
 - ~ 40,000 commits per workday



Types of Software Maintenance

- **Software maintenance:** modifications of software performed after delivery
- **Correction**
 - ✓ **Corrective maintenance:** Correct discovered faults
 - ✓ **Preventive maintenance:** Prevent occurrences of faults/failures (e.g., refactoring, document improvement)
- **Enhancement**
 - ✓ **Adaptive maintenance:** Adapt to a changed or changing environment
 - ✓ **Perfective maintenance:** Implement new or changed user requirements

Comparison of Maintenance Types

Maintenance Type	Request often from customer?	Effect of change visible to customers?	Need testing? (if yes, for what?)
Corrective	X	X	X (reproduce failure, confirm fix, not break)
Preventative			X (not break)
Adaptive		X?	X (confirm adaptation, not break)
Perfective	X	X	X (test new features, not break)

Summary

- Software testing
 - Aim
 - Test case
 - Mistake, fault, failure, error
 - Software testing types
 - White-box testing
 - Black-box testing
 - Software debugging (fault localization and defect repair)
 - Software maintenance